

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN



HCMUTE

**ỨNG DỤNG VOLATILITY 3 TRONG
TRÍCH XUẤT IOC TỪ MEMORY DUMP
CỦA MÃ ĐỘC**

Mã môn: MAAN431680

Ngành: AN TOÀN THÔNG TIN

GVHD : Th.S Đinh Công Đoàn

**SVTH : Huỳnh Văn Đức An -
2216200**

**SVTH : Trần Đông Phương -
22162034**

Thành phố Hồ Chí Minh, tháng 2 năm 2025

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN



HCMUTE

ỨNG DỤNG VOLATILITY 3 TRONG TRÍCH XUẤT IOC TỪ MEMORY DUMP CỦA MÃ ĐỘC

Mã môn: MAAN431680

Ngành: AN TOÀN THÔNG TIN

GVHD : Th.S Đinh Công Đoan

**SVTH : Huỳnh Văn Đức An -
2216200**

**SVTH : Trần Đông Phương -
22162034**

Thành phố Hồ Chí Minh, tháng 2 năm 2025

KHỐI LƯỢNG CÔNG VIỆC

Huỳnh Văn Đức An	Tối ưu sản phẩm, sửa lỗi, cải tiến pipeline	100%
Trần Đông Phương	Nghiên cứu pipeline, cách hoạt động, xây dựng khung sườn của sản phẩm	100%

NHẬN XÉT CỦA GIẢNG VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Giáo viên hướng dẫn

(Ký và ghi rõ họ tên)

MỤC LỤC

PHẦN 1 MỞ ĐẦU	1
1. Tóm tắt	1
2. Đặt vấn đề	1
2.1. Tóm lược nghiên cứu	1
2.2. Tính cấp thiết	2
2.3. Một số tài liệu liên quan	2
2.4. Lý do chọn đề tài	3
2.5. Mục tiêu đề tài	3
2.7. Phương pháp nghiên cứu	4
2.8. Nội dung đề tài	4
PHẦN 2 NỘI DUNG	5
Chương 1: Giới thiệu	5
1.1. Giới thiệu bài toán	5
1.2. Tổng quan về mã độc hiện đại và xu hướng tấn công qua bộ nhớ	5
1.3. Định nghĩa Indicator of Compromise (IoC)	6
1.4. Giới thiệu Volatility3	7
1.5. Giới thiệu Model Context Protocol (MCP)	8
1.6. Tổng quan hệ thống Automatic Volatility3 Pipeline IOC Extraction with AI Agent	9
Chương 2 : Phân tích yêu cầu	11
2.1 Yêu cầu đề tài	11
2.2 Đề xuất giải pháp	11
2.2.1 Hướng tiếp cận	11
2.2.2 Kiến trúc pipeline đề xuất	12
2.2.3 Vai trò của AI Agent và MCP	13
2.3 Sơ đồ nguyên lý pipeline hệ thống	13
2.3.1 Kiến trúc tổng thể	13
2.3.2 Sơ đồ luồng xác minh IOC	14
2.3.3 Sơ đồ trích xuất IOC ba tầng	15
2.4 Nguyên tắc làm việc	16
2.4.1 Giai đoạn 1, Khởi tạo và phát hiện môi trường	16
2.4.2 Giai đoạn 2, Lập kế hoạch phân tích	17
2.4.3 Giai đoạn 3, Thực thi plugin song song	17
2.4.4 Giai đoạn 4, Trích xuất IOC ba tầng	18
2.4.5 Giai đoạn 5, Xác minh và tính điểm	18

2.4.6 Giai đoạn 6, Sinh báo cáo.....	19
2.5 Công nghệ và giải pháp.....	19
2.5.1 Volatility3, Framework phân tích bộ nhớ.....	19
2.5.2 Model Context Protocol (MCP), Giao thức kết nối AI Agent.....	19
2.5.3 VirusTotal và AbuseIPDB, Threat Intelligence.....	20
2.5.4 Redis, Bộ nhớ đệm phân tán.....	20
2.5.5 Docker Compose, Triển khai và đóng gói.....	21
2.6 Tổng kết.....	21
Chương 3 : Kiến thức nền tảng.....	22
3.1 Phân tích bộ nhớ tiến trình Windows.....	22
3.1.1 VAD tree và quyền bảo vệ trang nhớ.....	22
3.1.2 EPROCESS linked list và pool scanning.....	23
3.2 Symbol Table và ISF trong Volatility3.....	24
3.3 MITRE ATT&CK và thiết kế quy tắc phát hiện.....	27
3.4 Cơ chế scoring VirusTotal và AbuseIPDB.....	30
3.5 Model Context Protocol (MCP).....	33
3.6 Caching strategy: TTL và two-level cache.....	34
Chương 4 : ỨNG DỤNG.....	38
4.1 Môi trường thực nghiệm.....	38
4.1.1 Máy ảo windows 11.....	38
4.1.2 Máy ảo Kali linux.....	39
4.1.3 Môi trường phân tích.....	41
4.2 Mẫu malware được chọn: SmokeLoader.....	41
4.3 Thu thập bằng chứng hành vi.....	42
4.3.1 Process Monitor.....	42
4.3.2 Wireshark.....	45
4.4 Chạy pipeline.....	46
4.5 Phân tích kết quả và đối chiếu.....	50

PHẦN 1 MỞ ĐẦU

1. Tóm tắt

Phân tích mã độc (malware analysis) là lĩnh vực nghiên cứu nhằm hiểu rõ hành vi, cơ chế hoạt động và mức độ nguy hiểm của các chương trình độc hại. Trong khi phân tích tĩnh (static analysis) và phân tích động trong sandbox đã được sử dụng rộng rãi, một lớp mã độc ngày càng phổ biến, đặc biệt là fileless malware và các kỹ thuật in-memory injection, lại không để lại dấu vết trên đĩa cứng, khiến hai phương pháp trên gần như bất lực. Phân tích bộ nhớ RAM (memory analysis) ra đời để lấp khoảng trống này : bằng cách kiểm tra trực tiếp nội dung bộ nhớ tại thời điểm mã độc đang chạy, người phân tích có thể quan sát hành vi thực sự của mã độc mà không bị che khuất bởi các kỹ thuật obfuscation hay packing.

Tuy nhiên, quy trình phân tích RAM dump bằng Volatility3 hiện nay vẫn chủ yếu thủ công, người phân tích phải biết chọn đúng plugin, đọc output thô và tự liên kết các artifact lại để nhận diện hành vi độc hại. Đề tài này xây dựng một hệ thống tự động hóa quy trình đó: nhận vào file dump RAM từ máy bị nhiễm, tự động chạy 18 plugin Volatility3 theo hai nhóm network và host, trích xuất Indicators of Compromise (IOC), xác minh qua VirusTotal và AbuseIPDB, rồi sinh báo cáo phân tích có cấu trúc. Toàn bộ pipeline được thiết kế và triển khai dưới dạng MCP Server, bao gồm việc xây dựng từng tool từ đầu : từ cơ chế nhận diện OS, lập lịch chạy song song 18 plugin Volatility3, đến tầng extraction phân loại IOC theo ngữ cảnh plugin nguồn, tầng validation tích hợp VirusTotal và AbuseIPDB với Redis cache, và cuối cùng là engine sinh báo cáo có MITRE ATT&CK mapping. Giao thức MCP được chọn vì cho phép AI agent tương tác với pipeline đã xây dựng theo từng bước có kiểm soát.

2. Đặt vấn đề

2.1. Tóm lược nghiên cứu

Phân tích bộ nhớ trong nghiên cứu mã độc đã được đặt nền tảng từ công trình của Ligh và cộng sự (2014) trong *The Art of Memory Forensics*, tài liệu hệ thống hóa toàn bộ kỹ thuật khai thác cấu trúc bộ nhớ Windows để phát hiện mã độc, từ process injection, DKOM ẩn tiến trình, đến rootkit trong kernel. Đây vẫn là tài liệu tham chiếu cơ bản cho hầu hết các nghiên cứu về memory-based malware analysis sau này.

Các nghiên cứu tiếp theo mở rộng theo hướng tự động hóa phát hiện mã độc từ dump RAM. Kolosnjaji và cộng sự (được trích dẫn trong Al-Shaer et al., 2024) lập luận rằng phân tích tĩnh và phân tích hành vi thông thường ngày càng bất lực trước các dòng mã độc hiện đại, và đề xuất chuyển hướng sang phân tích bộ nhớ volatile như một chiến lược đáng tin cậy hơn. Nghiên cứu của Mohaisen và cộng sự (được trích dẫn trong Alzahrani & Alqahtani, 2022) kết hợp memory forensics với dynamic analysis, đưa kết

quả đặc trưng vào classifier học máy và đạt độ chính xác 98.50% trên tập mẫu kiểm thử, cho thấy tiềm năng lớn khi xử lý artifact từ bộ nhớ một cách có hệ thống.

Về hướng trích xuất IOC từ bộ nhớ, Bayer và cộng sự tại DTU (2021) là một trong số ít nghiên cứu trực tiếp tự động hóa việc phân tích IOC từ Volatility plugin output, đề xuất kiến trúc interfacing với Volatility để gọi plugin và trích xuất đặc trưng IOC tự động. Gần đây hơn, Angeloska và cộng sự (2025) đề xuất pipeline kết hợp LLM với human-in-the-loop để trích xuất IOC từ threat report, giảm 43% thời gian so với annotation thủ công, minh chứng cho xu hướng tích hợp LLM vào quy trình phân tích IOC đang được nghiên cứu cộng đồng.

Điểm chung của các nghiên cứu trên là vẫn xử lý từng bước rời rạc: hoặc tập trung vào detection, hoặc vào classification, hoặc vào IOC extraction, chưa có công trình nào xây dựng pipeline khép kín từ dump RAM đến IOC có validation và báo cáo cuối cùng theo chuẩn MITRE ATT&CK. Đề tài này lấp khoảng trống đó, đồng thời bổ sung lớp giao tiếp MCP để AI agent có thể điều phối toàn bộ quy trình.

2.2. Tính cấp thiết

Mã độc hiện đại ngày càng tinh vi hơn trong việc né tránh phát hiện. Một trong những xu hướng nổi bật nhất là fileless malware, loại mã độc không ghi file thực thi nào xuống đĩa mà hoạt động hoàn toàn trong bộ nhớ RAM. Cobalt Strike beacon, công cụ post-exploitation phổ biến trong các chiến dịch APT, thường được inject trực tiếp vào tiến trình hợp lệ như explorer.exe hay svchost.exe thông qua kỹ thuật reflective loading. Emotet và TrickBot sử dụng process hollowing để thay thế nội dung tiến trình hợp lệ bằng shellcode độc hại trong khi vẫn giữ nguyên tên tiến trình. Với những kỹ thuật này, phân tích tĩnh trên đĩa sẽ không tìm thấy gì, chỉ có dump RAM mới tiết lộ payload thực sự đang chạy.

Ngoài ra, các kỹ thuật persistence qua registry (Run key, Winlogon, AppInit_DLLs), credential dumping từ LSASS, và C2 communication ẩn trong HTTPS traffic đều để lại dấu vết rõ ràng trong bộ nhớ, nhưng chỉ có thể phát hiện nếu biết cách khai thác đúng artifact từ dump RAM. Việc nhận diện và trích xuất các dấu vết này một cách có hệ thống chính là bài toán mà đề tài đặt ra.

2.3. Một số tài liệu liên quan

Đề tài được xây dựng dựa trên và tham chiếu từ các nguồn sau:

- Volatility3 Official Documentation (volatility3.readthedocs.io): tài liệu kỹ thuật mô tả kiến trúc plugin, symbol table system, và cách từng plugin khai thác cấu trúc nội bộ của Windows để trích xuất artifact.
- The Art of Memory Forensics (Ligh, Case, Levy, Walters, Wiley, 2014): tài liệu học thuật nền tảng về kỹ thuật phân tích bộ nhớ, phát hiện mã độc ẩn và các kỹ thuật evasion trong RAM.

- MITRE ATT&CK Framework (attack.mitre.org): framework phân loại kỹ thuật tấn công theo tactics và techniques, dùng làm cơ sở để mapping từng IOC trích xuất được với hành vi tấn công cụ thể của mã độc.
- VirusTotal API v3: nền tảng tra cứu hash file, địa chỉ IP và domain qua hơn 70 antivirus engine, tích hợp Redis cache để giảm thiểu số lần gọi API trùng lặp.
- AbuseIPDB API: dịch vụ tra cứu reputation địa chỉ IP dựa trên lịch sử báo cáo từ cộng đồng, hỗ trợ phát hiện địa chỉ C2 đã được biết đến.
- Model Context Protocol Specification (Anthropic, 2024): giao thức chuẩn hóa giao tiếp giữa LLM và external tool, là nền tảng kỹ thuật để AI agent điều phối pipeline phân tích.

2.4. Lý do chọn đề tài

Volatility3 hiện là framework memory analysis mạnh nhất và được duy trì tích cực nhất trong cộng đồng bảo mật. Với 18 plugin bao phủ từ phát hiện process injection (malfind, hollowprocesses), hidden process (psscan so với pslist), registry persistence (printkey, hivelist), đến credential artifact (ldrmodules, amcache), Volatility3 cung cấp đủ dữ liệu để dựng lại hành vi của mã độc trong bộ nhớ.

Điểm chưa được giải quyết là: sau khi chạy plugin, người phân tích vẫn phải tự đọc hàng nghìn dòng output thô để tìm artifact có ý nghĩa. Đề tài lấp khoảng trống này bằng cách xây dựng tầng extraction tự động (phân loại IOC network/host, context-aware confidence scoring) và tầng validation (VirusTotal, AbuseIPDB), rồi để AI agent điều phối toàn bộ, giúp người phân tích nhận kết quả có cấu trúc thay vì output thô.

2.5. Mục tiêu đề tài

Đề tài đặt ra ba mục tiêu cụ thể:

Thứ nhất, xây dựng pipeline tự động hóa hoàn chỉnh phục vụ phân tích mã độc qua RAM dump, từ nhận diện OS, chạy song song 18 plugin Volatility3, trích xuất và phân loại IOC, xác minh, đến sinh báo cáo, không cần can thiệp thủ công.

Thứ hai, phân loại IOC theo hai hướng rõ ràng phù hợp với đặc trưng hành vi mã độc: network-based IOC (địa chỉ IP C2, domain độc hại, kết nối từ tiến trình bất thường) và host-based IOC (memory injection, registry persistence, hidden process, credential dumping artifact). Mỗi IOC được gán MITRE ATT&CK technique tương ứng để hỗ trợ phân tích hành vi.

Thứ ba, tích hợp AI agent làm lớp điều phối thông qua MCP, người phân tích chỉ cần mô tả yêu cầu bằng ngôn ngữ tự nhiên, agent tự quyết định thứ tự gọi tool, xử lý kết quả trung gian và trả về báo cáo cuối cùng.

2.6. Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu là file memory dump định dạng raw (.raw, .mem) thu thập từ hệ thống Windows, tập trung vào Windows 10 và Windows 11 64-bit, cũng như các

phiên bản cũ hơn (Windows 7, 8, 8.1) vẫn còn xuất hiện trong môi trường doanh nghiệp thực tế.

Phạm vi tập trung vào các nhóm malware phổ biến: ransomware, trojan, fileless malware và lateral movement..

Đề tài không bao gồm: thu thập bộ nhớ trực tiếp từ máy đang chạy (live acquisition), phân tích bộ nhớ thiết bị di động, hay phân tích Linux dump.

2.7. Phương pháp nghiên cứu

Nghiên cứu tài liệu, tổng hợp từ tài liệu kỹ thuật chính thống của Volatility3, MITRE ATT&CK, và MCP Specification. Đây là bước nền để xác định plugin cần dùng, artifact cần thu thập, và rule detection cần xây dựng trong RegistryAnalyzer.

Thực nghiệm, triển khai hệ thống qua Docker Compose và kiểm thử trực tiếp trên dump mẫu thực tế. Đánh giá dựa trên số lượng IOC phát hiện, tỉ lệ false positive, và thời gian xử lý so với phân tích thủ công.

Xây dựng và đánh giá, từng module (ExtractionPipeline, ValidationPipeline, RegistryAnalyzer) được thiết kế theo nguyên tắc modular, kiểm thử độc lập trước khi tích hợp. Pipeline end-to-end được kiểm tra qua script e2e_test.py chạy trực tiếp trong container.

2.8. Nội dung đề tài

MCP Server triển khai sáu tool theo chuỗi pipeline: detect_os, run_plugins (chạy song song 18 plugin, phân tách network_data/host_data), run_plugin (chạy đơn lẻ một plugin), ioc_extract_from_store, ioc_validate (VirusTotal + AbuseIPDB + Whitelist, cache Redis), và generate_report.

ExtractionPipeline gồm ba tầng song song: IOCExtractor (regex: IP, hash, domain, path), ContextAwareExtractor (structured extraction từ malfind/netscan/svcscan), và RegistryAnalyzer (26 rule MITRE-mapped bao phủ persistence, defense evasion, credential access, execution).

ValidationPipeline xác minh IOC qua WhitelistValidator (private IP, known-good domain, system process), VirusTotalValidator và AbuseIPDBValidator (cache Redis để tránh gọi API trùng), và CorrelationGuard (hạ confidence các finding đơn lẻ không có corroboration).

Đánh giá thực nghiệm trên bộ mẫu thực tế, đo lường khả năng phát hiện IOC và so sánh với kết quả phân tích thủ công.

PHẦN 2 NỘI DUNG

Chương 1: Giới thiệu

1.1. Giới thiệu bài toán

Trong quy trình phân tích mã độc, việc xác định chính xác hành vi của một mẫu độc hại thường không thể chỉ dựa vào phân tích tĩnh hay chạy thử trong môi trường cô lập. Phân tích tĩnh, dù hiệu quả trong việc đọc cấu trúc file thực thi, trích xuất chuỗi ký tự hay dịch ngược mã nguồn, hoàn toàn bất lực khi mã độc được đóng gói, mã hóa nhiều lớp, hoặc sử dụng kỹ thuật biến hình để thay đổi chữ ký liên tục. Phân tích động trong môi trường sandbox cải thiện điều này bằng cách quan sát hành vi thực thi, nhưng nhiều dòng mã độc hiện đại có khả năng nhận biết môi trường ảo hóa và tự ngừng hoạt động, khiến kết quả thu được không phản ánh hành vi thực sự khi chạy trên máy nạn nhân.

Phân tích bộ nhớ RAM ra đời để lấp khoảng trống này. Khi một tiến trình đang chạy, dù ban đầu bị đóng gói hay mã hóa đến đâu, phần mã thực thi thực sự buộc phải được giải nén và nạp vào bộ nhớ trước khi bộ xử lý có thể thực hiện. Điều đó có nghĩa là tại thời điểm chụp bộ nhớ, toàn bộ trạng thái thực của mã độc đang hiện diện: mã độc đã được giải nén hoàn toàn, các kết nối mạng đang mở, khóa mã hóa còn lưu trong vùng heap, thông tin xác thực vừa bị đánh cắp từ tiến trình quản lý phiên đăng nhập. Đây là dữ liệu không thể tìm thấy ở bất kỳ nguồn nào khác, không có trên ổ đĩa, không có trong nhật ký hệ thống, và biến mất hoàn toàn khi máy bị tắt nguồn.

Tuy nhiên, khai thác dữ liệu từ bản sao bộ nhớ không đơn giản. Một file dump thô là dãy nhị phân phẳng đại diện cho toàn bộ nội dung bộ nhớ tại thời điểm chụp, không có cấu trúc rõ ràng nếu không có công cụ phân tích phù hợp. Người phân tích cần biết cách duyệt danh sách tiến trình qua cấu trúc nội bộ của hệ điều hành, tìm vùng nhớ có quyền truy cập đáng ngờ, hay truy vết dữ liệu registry còn lưu trong bộ nhớ. Volatility3 là bộ khung phân tích được xây dựng chính xác cho mục đích này. Bài toán đề tài đặt ra là: tự động hóa quy trình chạy các plugin phân tích, trích xuất các dấu hiệu nhận dạng mối đe dọa, xác minh và tổng hợp thành báo cáo phân tích hoàn chỉnh mà không cần người phân tích thao tác thủ công từng bước.

1.2. Tổng quan về mã độc hiện đại và xu hướng tấn công qua bộ nhớ

Các dòng mã độc đời đầu hoạt động theo cơ chế đơn giản: ghi file thực thi xuống ổ đĩa, thiết lập cơ chế tự khởi động qua registry, rồi liên lạc với máy chủ điều khiển. Cơ chế này dễ phát hiện bằng các giải pháp diệt virus thông thường và dễ truy vết vì để lại dấu vết rõ ràng trên hệ thống tập tin. Tuy nhiên, từ khoảng năm 2017 trở đi, xu hướng mã độc không để lại file (fileless malware) và lạm dụng công cụ hợp lệ của hệ thống ngày càng chiếm ưu thế, đặc trưng bởi việc không ghi bất kỳ tệp thực thi độc hại nào xuống ổ đĩa, thay vào đó lạm dụng các tiện ích sẵn có của Windows như trình thông dịch lệnh, dịch vụ

quản lý hệ thống, hay các công cụ chứng chỉ mạng để thực thi mã độc trực tiếp trong bộ nhớ.

Số liệu thống kê phản ánh rõ xu hướng này: theo ReliaQuest, 86,2% các sự cố nghiêm trọng được ghi nhận năm 2023 có liên quan đến mã độc không để lại file. Nghiên cứu của Viện Ponemon chỉ ra rằng dạng tấn công này có khả năng thành công cao hơn gấp 10 lần so với tấn công truyền thống dựa trên file.

Về mặt kỹ thuật, các phương pháp tấn công qua bộ nhớ phổ biến nhất bao gồm:

- Tiêm mã vào tiến trình (Process Injection, T1055): Chèn đoạn mã thực thi hoặc thư viện liên kết động vào tiến trình hợp lệ đang chạy như explorer.exe hay svchost.exe. Tiến trình bị tiêm vẫn hiển thị tên bình thường nhưng thực thi mã của kẻ tấn công, kỹ thuật này được sử dụng rộng rãi trong các bộ công cụ tấn công chuyên nghiệp.
- Rỗng hóa tiến trình (Process Hollowing, T1055.012): Tạo một tiến trình hợp lệ ở trạng thái tạm dừng, xóa nội dung gốc, rồi thay bằng mã độc trước khi tiếp tục thực thi. Kỹ thuật này được ghi nhận trong nhiều biến thể của các dòng mã độc ngân hàng như TrickBot và Emotet.
- Nạp thư viện phản chiếu (Reflective DLL Loading): Nạp thư viện liên kết động trực tiếp từ bộ nhớ mà không ghi xuống ổ đĩa hay đăng ký qua trình nạp module của Windows, do đó không xuất hiện trong danh sách module được nạp chính thức, chỉ có thể phát hiện khi kiểm tra bộ nhớ bằng plugin chuyên biệt.
- Lạm dụng công cụ hệ thống hợp lệ: Sử dụng trình thông dịch lệnh với tham số thực thi mã được mã hóa Base64, hay dùng các tiện ích cập nhật, chứng chỉ mạng của hệ điều hành để tải và chạy mã độc từ mạng, toàn bộ đều là chương trình hợp lệ của Windows, không bị phần mềm bảo mật ngăn chặn.

Điểm chung của các kỹ thuật trên là dấu vết duy nhất tồn tại trong bộ nhớ RAM, và biến mất hoàn toàn khi máy tắt. Điều đó đặt ra yêu cầu phải thu thập và phân tích bộ nhớ sớm nhất có thể sau khi phát hiện dấu hiệu xâm phạm.

1.3. Định nghĩa Indicator of Compromise (IoC)

Chỉ số nhận dạng mối đe dọa (Indicators of Compromise, IOC) là các dấu hiệu kỹ thuật có thể quan sát được, cho thấy một hệ thống có khả năng đã bị xâm phạm hoặc đang chịu tác động của mã độc. Khác với các chỉ số chiến thuật cấp cao mô tả cách thức và phương pháp tấn công, các chỉ số nhận dạng thường là giá trị cụ thể và có thể đo đếm được, địa chỉ máy chủ điều khiển, giá trị băm của file độc hại, khóa registry được dùng để tạo cơ chế tự khởi động, hay chuỗi ký tự đặc trưng trong bộ nhớ tiến trình.

Mô hình Pyramid of Pain (Bianco, 2013) phân cấp các chỉ số nhận dạng theo mức độ khó thay đổi từ phía kẻ tấn công, giá trị băm file là chỉ số dễ thay đổi nhất (chỉ cần biên dịch lại là chữ ký thay đổi hoàn toàn), trong khi phương thức và kỹ thuật tấn công là

lớp khó thay đổi nhất vì phản ánh tư duy và quy trình của kẻ tấn công. Việc kết hợp nhiều loại chỉ số nhận dạng khác nhau giúp tăng độ tin cậy của kết quả phân tích.

Trong ngữ cảnh phân tích bộ nhớ RAM, chỉ số nhận dạng được phân thành hai nhóm chính:

Network-based IOC là các dấu hiệu liên quan đến hoạt động kết nối mạng của mã độc, trích xuất chủ yếu từ kết quả quét kết nối mạng và xử lý tài nguyên trong bộ nhớ:

- Địa chỉ IP của máy chủ điều khiển mà mã độc đang kết nối hoặc đang lắng nghe
- Tên miền độc hại được phân giải từ bộ nhớ tiến trình
- Kết nối mạng từ tiến trình bất thường, ví dụ tiến trình soạn thảo văn bản đang mở kết nối ra ngoài mạng
- Cổng kết nối không chuẩn được dùng để qua mặt tường lửa

Host-based IOC là các dấu hiệu liên quan đến hành vi trực tiếp trên hệ thống, phong phú hơn nhiều và là trọng tâm chính khi phân tích bộ nhớ:

- Vùng nhớ có quyền đọc-ghi-thực thi chứa tiêu đề tệp thực thi, dấu hiệu điển hình của tiêm mã vào tiến trình
- Tiến trình xuất hiện khi quét toàn bộ vùng nhớ nhưng vắng mặt trong danh sách tiến trình hệ thống, kỹ thuật ẩn tiến trình qua can thiệp cấu trúc nhân
- Thư viện liên kết động không có trong danh sách module được nạp chính thức, dấu hiệu của kỹ thuật nạp phản chiếu
- Khóa registry thiết lập cơ chế tự khởi động còn trong vùng nhớ hive
- Dữ liệu xác thực bị trích xuất từ tiến trình quản lý phiên đăng nhập
- Giá trị băm của tệp thực thi được nạp từ đường dẫn đáng ngờ như thư mục tạm, thư mục dữ liệu ứng dụng

Việc phân biệt hai nhóm này không chỉ có giá trị phân loại mà còn quyết định hướng xử lý tiếp theo. Chỉ số theo hướng mạng giúp xác định cơ sở hạ tầng của kẻ tấn công và phong tỏa ở tầng mạng, trong khi chỉ số theo hướng máy chủ giúp tái dựng lại chuỗi hành vi của mã độc và đánh giá phạm vi ảnh hưởng trên hệ thống bị nhiễm.

1.4. Giới thiệu Volatility3

Volatility là bộ khung phân tích bộ nhớ mã nguồn mở được phát triển bởi Volatility Foundation, lần đầu ra mắt vào năm 2007 dưới tên gọi Volatility2. Trong nhiều năm, đây là công cụ tiêu chuẩn trong cộng đồng phân tích mã độc và điều tra số, được giảng dạy trong các khóa chứng chỉ bảo mật như SANS FOR508 và sử dụng rộng rãi trong các cuộc thi Capture The Flag. Tuy nhiên, kiến trúc của Volatility2 tích lũy nhiều hạn chế theo thời gian: hệ thống *profile* cứng buộc người phân tích phải biết chính xác phiên bản hệ điều hành của dump file và cài đặt profile phù hợp, điều này trở nên ngày càng khó khăn khi Windows liên tục cập nhật, kéo theo sự thay đổi về offset của các cấu trúc nội bộ.

Volatility3 được viết lại hoàn toàn từ đầu và ra mắt chính thức năm 2019, giải quyết vấn đề trên thông qua hệ thống *bảng ký hiệu* (symbol tables). Thay vì hardcode offset theo từng phiên bản, Volatility3 đọc thông tin gỡ lỗi chính thức từ Microsoft (thông qua file PDB) để tự động xác định vị trí chính xác của từng cấu trúc trong bộ nhớ, ngay cả với các bản vá cập nhật nhỏ nhất của Windows.

Điểm khác biệt cốt lõi giữa hai phiên bản có thể tóm gọn như sau:

Đặc điểm	Volatility2	Volatility3
Nhận diện OS	Profile cứng, chỉ định thủ công	Tự động qua bảng ký hiệu
Cập nhật Windows mới	Phải tạo profile mới	Tải bảng ký hiệu tự động từ Microsoft
Cú pháp plugin	pslist	windows.pslist.PsList (có namespace)
Kiến trúc	Monolithic	Phân lớp, hỗ trợ nhiều định dạng file
Python	Python 2	Python 3

Về kiến trúc, Volatility3 tổ chức plugin theo không gian tên phân cấp, ví dụ windows.malfind.Malfind, windows.registry.printkey.PrintKey, giúp phân biệt rõ ràng giữa plugin dành cho Windows và Linux. Mỗi plugin nhận đầu vào là các lớp bộ nhớ (memory layers) đã được trừu tượng hóa, truy vấn các cấu trúc nội bộ thông qua bảng ký hiệu, rồi trả về kết quả dạng lưới có cấu trúc (TreeGrid), thuận tiện để xử lý tự động.

1.5. Giới thiệu Model Context Protocol (MCP)

Model Context Protocol (MCP) là một giao thức mã nguồn mở do Anthropic đề xuất và công bố vào cuối năm 2024, với mục tiêu chuẩn hóa cách thức các mô hình ngôn ngữ lớn kết nối và tương tác với các hệ thống bên ngoài. Về bản chất, MCP giải quyết một bài toán thực tế: mỗi mô hình ngôn ngữ trước đây phải tích hợp thủ công với từng công cụ bên ngoài theo cách riêng, không có chuẩn chung, dẫn đến việc mỗi tích hợp phải xây dựng lại từ đầu. MCP lấy cảm hứng từ Language Server Protocol trong lập trình, giao thức đã thành công trong việc chuẩn hóa tích hợp giữa trình soạn thảo mã nguồn và các ngôn ngữ lập trình.

Kiến trúc MCP gồm ba thành phần chính:

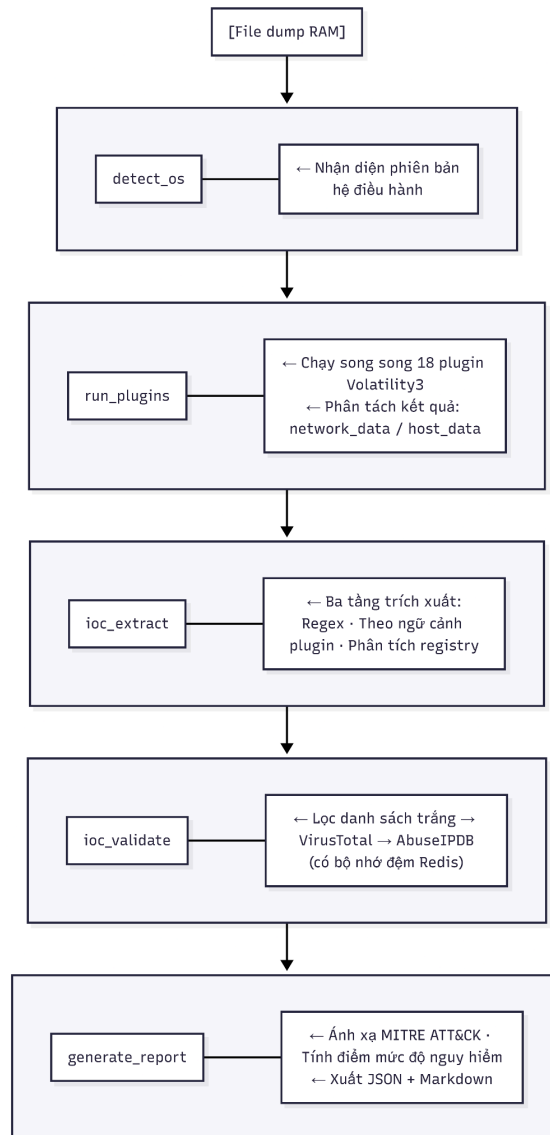
- MCP Host, ứng dụng chứa mô hình ngôn ngữ, nơi người dùng tương tác trực tiếp (ví dụ Claude Desktop, Cline trong VSCode)
- MCP Client, bộ phận tích hợp bên trong Host, chịu trách nhiệm kết nối với các MCP Server và dịch giữa yêu cầu của mô hình và giao thức MCP
- MCP Server, dịch vụ độc lập cung cấp các công cụ (tools), dữ liệu (resources), hoặc hướng dẫn (prompts) cho mô hình sử dụng

Giao tiếp giữa Client và Server sử dụng JSON-RPC 2.0, định dạng nhẹ, dễ triển khai qua nhiều môi trường khác nhau (stdio, HTTP). Khi mô hình ngôn ngữ nhận một yêu cầu từ người dùng và nhận thấy cần gọi công cụ bên ngoài, nó sinh ra lời gọi công cụ có cấu trúc, MCP Client chuyển tiếp lời gọi đó đến MCP Server phù hợp, nhận kết quả trả về, rồi đưa lại cho mô hình để tiếp tục xử lý.

Trong đề tài này, toàn bộ pipeline phân tích, từ nhận diện hệ điều hành, chạy plugin Volatility3, trích xuất chỉ số nhận dạng, xác minh đến sinh báo cáo, được đóng gói thành các công cụ của một MCP Server. Điều này có nghĩa là tất cả logic xử lý đều được xây dựng thủ công bên trong Server; MCP chỉ là lớp giao tiếp cho phép mô hình ngôn ngữ gọi đúng công cụ đúng thứ tự dựa trên ngữ cảnh phân tích, thay vì người dùng phải nhớ và gõ từng lệnh một.

1.6. Tổng quan hệ thống Automatic Volatility3 Pipeline IOC Extraction with AI Agent

Hệ thống được xây dựng gồm năm giai đoạn xử lý nối tiếp nhau, từ đầu vào là file bản sao bộ nhớ thô đến đầu ra là báo cáo phân tích có cấu trúc:



Toàn bộ hệ thống được đóng gói trong Docker Compose với hai container: mcp-server chạy pipeline chính và redis phục vụ bộ nhớ đệm kết quả tra cứu. Người phân tích kết nối thông qua bất kỳ MCP Client nào tương thích, Claude Desktop hay Cline trong VSCode, và tương tác với hệ thống bằng ngôn ngữ tự nhiên, trong khi toàn bộ logic phân tích chạy bên trong Server.

Chương 2 : Phân tích yêu cầu

2.1 Yêu cầu đề tài

Trong điều tra số (digital forensics), phân tích bộ nhớ RAM là một trong những kỹ thuật quan trọng nhất để phát hiện mã độc và truy vết hành vi tấn công. Khi một hệ thống bị nghi ngờ xâm phạm, điều tra viên thường thu thập file dump RAM, một bản sao toàn bộ trạng thái bộ nhớ tại thời điểm xảy ra sự cố, và tiến hành phân tích để tìm kiếm các chỉ số nhận dạng mối đe dọa (Indicators of Compromise, IOC).

Tuy nhiên, quy trình phân tích thủ công hiện tại tồn tại nhiều hạn chế nghiêm trọng. Điều tra viên phải thực thi từng plugin Volatility3 riêng lẻ, đọc và lọc thủ công hàng nghìn dòng output thô, sau đó tự phán đoán xem dữ liệu nào là đáng ngờ. Một ca phân tích đầy đủ trên một file dump có thể kéo dài nhiều giờ đồng hồ, kết quả phụ thuộc nhiều vào kinh nghiệm cá nhân và dễ bỏ sót các tín hiệu ẩn. Ngoài ra, khi cần tra cứu chéo IOC với các nguồn threat intelligence bên ngoài như VirusTotal hay AbuseIPDB, công việc lại càng tốn thời gian hơn.

Từ thực tế đó, yêu cầu của đề tài được xác định như sau : Xây dựng một hệ thống tự động hóa quy trình phân tích file dump RAM bằng Volatility3, có khả năng trích xuất IOC một cách có hệ thống, xác minh độ tin cậy của từng IOC và xuất báo cáo pháp y phục vụ điều tra số.

Cụ thể, hệ thống cần đáp ứng ba nhóm yêu cầu cốt lõi:

Thứ nhất, Tự động hóa hoàn toàn : Từ đầu vào là một file dump RAM và mục tiêu phân tích (ví dụ: phát hiện mã độc, điều tra sự cố,...), hệ thống phải tự động lựa chọn plugin phù hợp, thực thi, và xử lý kết quả.

Thứ hai, Trích xuất và xác minh IOC chính xác : Hệ thống phải nhận diện được đa dạng loại IOC, địa chỉ IP, tên miền, giá trị băm, đường dẫn file, lệnh độc hại, tiêm mã vào tiến trình, khóa registry đáng ngờ, đồng thời giảm thiểu dương tính giả (false positive) thông qua xác minh đa tầng.

Thứ ba, Khả năng tích hợp với AI Agent : Hệ thống phải có khả năng hoạt động như một công cụ mà AI Assistant có thể gọi và điều phối, mở ra khả năng phân tích hội thoại tự nhiên.

2.2 Đề xuất giải pháp

2.2.1 Hướng tiếp cận

Từ ba nhóm yêu cầu trên, nhóm đề xuất xây dựng hệ thống mang tên: ***Automatic Volatility3 Pipeline IOC Extraction with AI Agent***

Ý tưởng cốt lõi là tổ chức toàn bộ quy trình phân tích thành một pipeline tự động nhiều giai đoạn, trong đó mỗi giai đoạn có trách nhiệm rõ ràng và kết quả của giai đoạn trước là đầu vào cho giai đoạn tiếp theo. Pipeline này được bọc bên ngoài bởi một MCP

Server, cho phép AI Agent gọi và điều phối toàn bộ hoặc từng bước theo ngữ cảnh hội thoại.

2.2.2 Kiến trúc pipeline đề xuất

Pipeline được chia thành sáu giai đoạn liên tiếp như sau:

Giai đoạn 1, Khởi tạo và phát hiện môi trường : Nhận đầu vào là đường dẫn file dump và mục tiêu phân tích. Hệ thống tự động phát hiện hệ điều hành của dump (Windows hoặc Linux) bằng cách thử chạy plugin windows.info.Info và banners.Banners, từ đó xác định bộ plugin phù hợp.

Giai đoạn 2, Lập kế hoạch phân tích (Decision Engine) : Module DecisionEngine đọc profile plugin tương ứng với cặp (os_type, goal) từ cấu hình, trả về danh sách plugin được xếp theo mức độ ưu tiên (priority 1 → 2 → 3). Hệ thống hỗ trợ năm profile mặc định: quick_triage, malware_detection, incident_response, rootkit_hunt, network_forensics, có thể mở rộng qua file plugin_profiles.yaml mà không cần sửa code.

Giai đoạn 3, Thực thi plugin (Volatility Executor) : Module VolatilityExecutor thực thi các plugin bất đồng bộ qua asyncio, với tối đa 3 plugin chạy song song. Trước mỗi lần chạy, hệ thống kiểm tra cache Redis theo khóa vol3:{hash_dump}:{plugin}:{hash_args}. Nếu trùng cache, kết quả được trả về ngay lập tức mà không cần khởi động tiến trình Volatility3, giảm đáng kể thời gian phân tích lặp lại trên cùng một file dump.

Giai đoạn 4, Trích xuất IOC :

Đây là giai đoạn cốt lõi, gồm ba lớp trích xuất hoạt động song song:

- Lớp 1, Regex scan (IOCExtractor): Quét văn bản output thô của từng plugin bằng biểu thức chính quy, trích xuất 7 loại IOC. Mỗi loại có source_gate (chỉ áp dụng với plugin liên quan) và exclude (loại bỏ các giá trị biết chắc là vô hại).
- Lớp 2, Context-aware (ContextAwareExtractor): Phân tích dữ liệu có cấu trúc từ từng plugin, phát hiện các hành vi đặc trưng như cấp tiến trình cha-con bất thường, lệnh PowerShell mã hóa, vùng nhớ RWX có tiêu đề PE, kết nối mạng từ tiến trình đáng ngờ đến cổng hiểm.
- Lớp 3, Registry rules (RegistryAnalyzer): Áp dụng 18 quy tắc tĩnh phân loại theo bốn nhóm kỹ thuật MITRE: persistence, defense evasion, credential access, execution.

Giai đoạn 5, Xác minh IOC (Validation Pipeline) :

Mỗi IOC được xử lý qua chuỗi xác minh:

1. Whitelist, loại ngay nếu là IP/domain/hash/process hệ thống đã biết lành tính
2. VirusTotal + AbuseIPDB, tra cứu threat intelligence bên ngoài (khi có API key)
3. Correlation Guard, hạ mức cảnh báo xuống "suspicious" nếu chỉ có tín hiệu hành vi đơn lẻ mà không có bằng chứng hỗ trợ từ các nguồn khác

Điểm tin cậy cuối cùng được tính theo trọng số: VirusTotal 40%, AbuseIPDB 30%, Whitelist 30%. Ngưỡng phán quyết: $\geq 0.70 \rightarrow$ malicious, $0.40\text{--}0.69 \rightarrow$ suspicious, $< 0.40 \rightarrow$ benign.

Giai đoạn 6, Sinh báo cáo (Report Generator) : Mỗi case được lưu vào thư mục riêng theo định dạng CASE_{OS}_{timestamp}/. Hệ thống xuất đồng thời: SUMMARY.txt (tổng hợp có Threat Score 0–100), report.md (Markdown có bảng và hyperlink MITRE ATT&CK), iocs.json (dữ liệu máy đọc), timeline.txt, attack_chain.txt, behavior_analysis.txt, narrative.txt và raw output từng plugin trong thư mục plugins/.

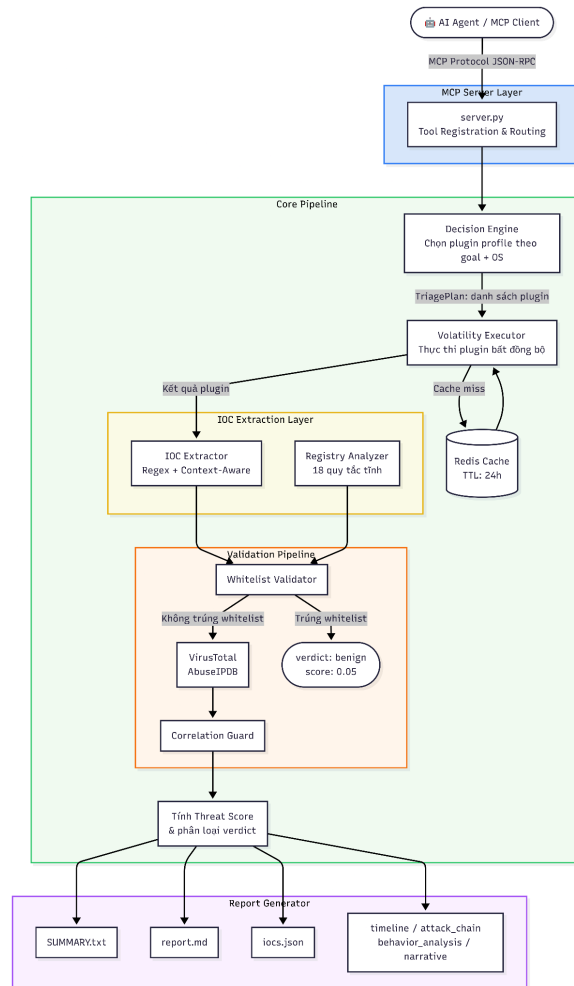
2.2.3 Vai trò của AI Agent và MCP

Toàn bộ pipeline trên được bọc bởi một MCP Server, một chuẩn giao thức mới cho phép AI Assistant (Claude, GPT...) tự khám phá và gọi các công cụ theo ngữ cảnh. Thay vì điều tra viên phải gõ lệnh thủ công từng bước, họ có thể tương tác bằng ngôn ngữ tự nhiên, AI Agent sẽ tự động gọi các tool MCP theo thứ tự phù hợp, diễn giải kết quả và trình bày tóm tắt cho người dùng, biến hệ thống từ một công cụ CLI thành một trợ lý điều tra số thông minh.

2.3 Sơ đồ nguyên lý pipeline hệ thống

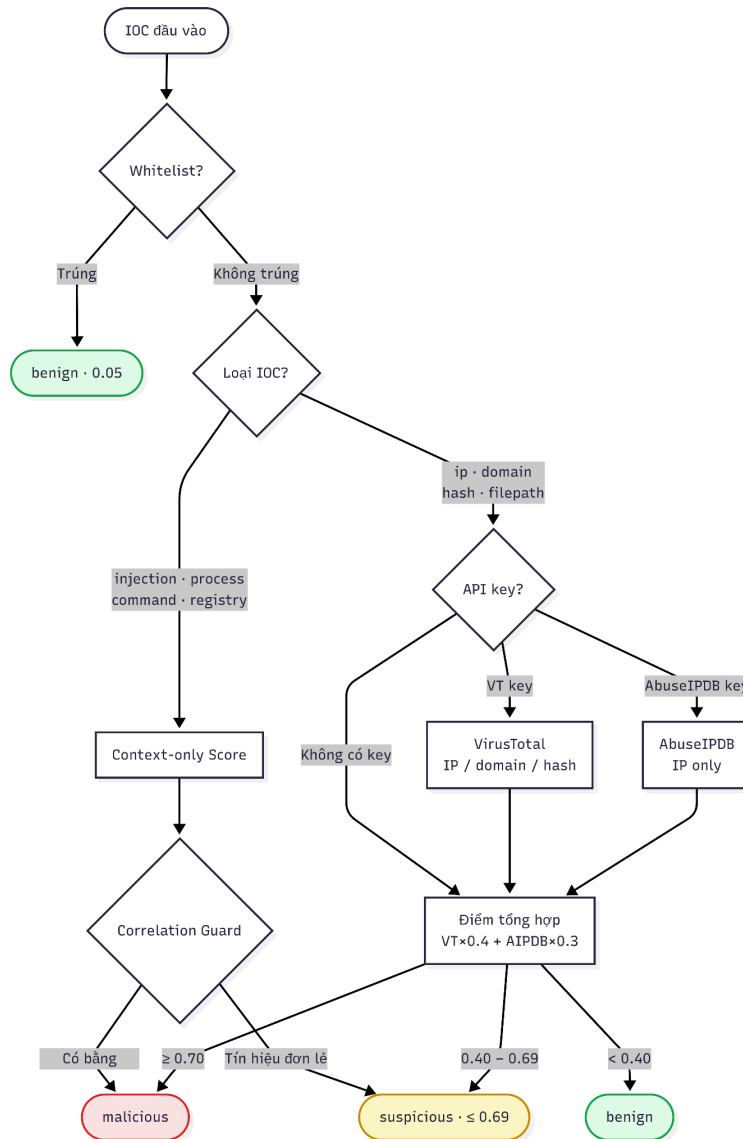
2.3.1 Kiến trúc tổng thể

Hệ thống được tổ chức theo mô hình pipeline tuyến tính nhiều tầng, trong đó đầu ra của mỗi tầng là đầu vào cho tầng kế tiếp. Toàn bộ pipeline được bọc bởi một MCP Server đóng vai trò giao tiếp với AI Agent bên ngoài.



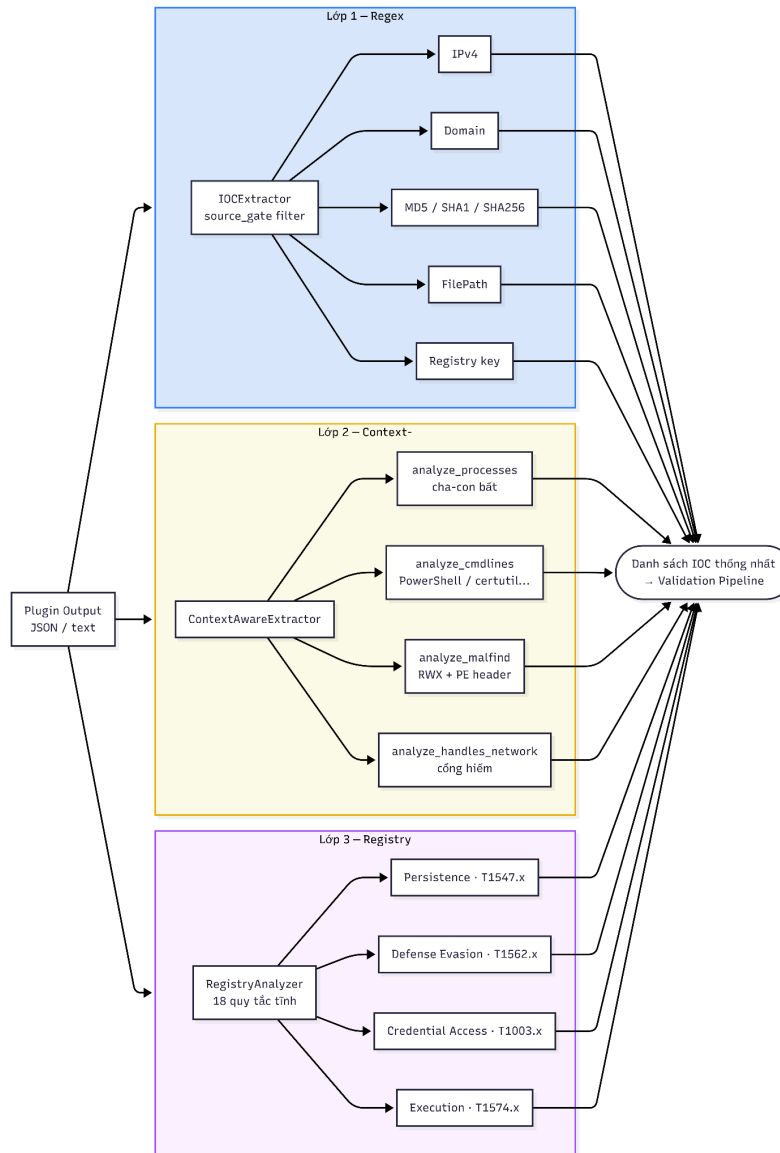
2.3.2 Sơ đồ luồng xác minh IOC

Giai đoạn xác minh IOC là giai đoạn có logic phức tạp nhất, với các nhánh phân kỳ theo loại IOC và cấu hình API.



2.3.3 Sơ đồ trích xuất IOC ba tầng

Giai đoạn trích xuất IOC được tổ chức thành ba lớp hoạt động độc lập và bổ sung cho nhau.



2.4 Nguyên tắc làm việc

Pipeline hoạt động theo sáu giai đoạn tuần tự, mỗi giai đoạn tương ứng với một module cụ thể trong hệ thống.

2.4.1 Giai đoạn 1, Khởi tạo và phát hiện môi trường

Khi nhận được yêu cầu phân tích, hệ thống bắt đầu bằng việc xác định hệ điều hành của file dump thông qua hai plugin thăm dò : windows.info.Info cho Windows và banners.Banners cho Linux. Kết quả xác định OS được truyền xuống tất cả các giai đoạn tiếp theo vì toàn bộ logic trích xuất, bộ plugin được chọn và các quy tắc phân tích đều phụ thuộc vào loại hệ điều hành. Đồng thời, một thư mục case mới được tạo theo định dạng CASE_{OS}_{timestamp}/ để lưu trữ toàn bộ kết quả phân tích.

2.4.2 Giai đoạn 2, Lập kế hoạch phân tích

Module DecisionEngine nhận cặp tham số (os_type, goal) và trả về một TriagePlan, đối tượng chứa danh sách các plugin cần thực thi cùng mức ưu tiên tương ứng. Danh sách plugin được tổ chức theo ba mức ưu tiên:

- Priority 1 (Required): Các plugin cốt lõi, bắt buộc phải chạy, có tác động trực tiếp đến kết quả chính của ca phân tích.
- Priority 2: Các plugin bổ sung, cung cấp thêm ngữ cảnh và tín hiệu hỗ trợ.
- Priority 3: Các plugin đặc thù, chỉ có giá trị trong một số trường hợp cụ thể.

Hệ thống mặc định hỗ trợ năm profile phân tích, với thời gian ước tính từ 4 đến 28 phút như bảng sau:

Profile	Mô tả	Windows (phút)
quick_triage	Đánh giá nhanh ban đầu	~5
malware_detection	Trích xuất IOC toàn diện	~18
incident_response	Thu thập đầy đủ: credentials, MFT, LSA	~28
rootkit_hunt	Phát hiện SSDT hook, driver ẩn	~20
network_forensics	Điều tra kết nối C2	~12

Cấu hình profile có thể được ghi đè hoàn toàn bằng file plugin_profiles.yaml bên ngoài mà không cần sửa mã nguồn.

2.4.3 Giai đoạn 3, Thực thi plugin song song

Module VolatilityExecutor thực thi các plugin bất đồng bộ thông qua asyncio.create_subprocess_exec, với tối đa 3 plugin chạy song song được kiểm soát bởi asyncio.Semaphore. Trước mỗi lần thực thi, hệ thống kiểm tra Redis theo khóa cache:

```
cache_key = "vol3:{sha256(100MB đầu dump)}:{plugin}:{md5(args)}"
```

Nếu trùng cache (TTL 24 giờ), kết quả được trả về ngay lập tức mà không cần khởi động tiến trình Volatility3. Cơ chế này đặc biệt hiệu quả trong các ca phân tích lặp lại hoặc khi cần chạy thêm plugin bổ sung trên cùng một dump. Riêng với plugin nặng như Malfind, timeout được nâng lên 1.800 giây để tránh ngắt sớm do thời gian quét bộ nhớ kéo dài.

2.4.4 Giai đoạn 4, Trích xuất IOC ba tầng

Sau khi có kết quả từ các plugin, ba lớp trích xuất được thực thi độc lập và kết quả được gộp lại thành danh sách IOC thống nhất.

Lớp 1, Regex scan: IOCEXtractor áp dụng biểu thức chính quy lên văn bản output thô. Mỗi loại IOC có source_gate, tập hợp plugin mà mẫu này mới có giá trị, tránh trích xuất vô nghĩa từ plugin không liên quan. Ví dụ, địa chỉ IP chỉ được trích xuất từ netscan, netstat, sockstat; giá trị băm chỉ từ amcache, filescan. Mỗi IOC được gán điểm tin cậy ban đầu theo loại và plugin nguồn.

Lớp 2, Context-aware: ContextAwareExtractor làm việc với dữ liệu có cấu trúc JSON từ từng plugin, phát hiện bốn nhóm hành vi: (1) cập tiến trình cha-con bất thường theo 16 mẫu định nghĩa cho Windows; (2) lệnh độc hại trong command line theo 19 mẫu regex được ánh xạ với kỹ thuật MITRE; (3) vùng nhớ RWX từ Malfind, chỉ lấy vùng có bảo vệ PAGE_EXECUTE_READ(WRITE), điểm tin cậy tăng lên 0.90 nếu có tiêu đề PE (MZ); (4) kết nối mạng từ tiến trình đáng ngờ đến cổng hiểm như 4444, 1337, 31337.

Lớp 3, Registry rules: RegistryAnalyzer áp dụng 18 quy tắc tĩnh với ba tầng khớp tuần tự: tên khóa → tên giá trị (tùy chọn) → nội dung dữ liệu (tùy chọn). Kết quả được loại trùng theo khóa tổng hợp {key}|{value}|{mitre_id} và sắp xếp theo mức độ nghiêm trọng từ critical xuống low.

2.4.5 Giai đoạn 5, Xác minh và tính điểm

Mỗi IOC được xử lý tuần tự qua ba lớp xác minh. Đầu tiên, WhitelistValidator kiểm tra: nếu trùng, IOC nhận điểm 0.05 và kết thúc pipeline ngay. Với các IOC loại hành vi thuần túy (injection, process, command, registry), điểm được tính trực tiếp từ ngữ cảnh mà không cần tra cứu bên ngoài. Với các IOC có giá trị tra cứu được (IP, domain, hash), VirusTotalValidator và AbuseIPDBValidator được gọi song song khi API key được cấu hình.

Khi điểm VirusTotal đạt từ 0.40 trở lên, trọng số VT được nâng lên 0.50 để phản ánh độ tin cậy cao hơn của kết quả đa engine.

Sau khi xử lý xong toàn bộ, CorrelationGuard kiểm tra lại: IOC hành vi bị đánh "malicious" nhưng không có IOC bằng chứng (IP/domain/hash) nào khác được xác nhận, không có kỹ thuật MITRE lặp lại ít nhất 2 lần, và không có xác nhận từ VT/AbuseIPDB,

sẽ tự động bị hạ xuống "suspicious" với điểm tối đa 0.69. Ngưỡng phán quyết cuối: $\geq 0.70 \rightarrow$ malicious, $0.40-0.69 \rightarrow$ suspicious, $< 0.40 \rightarrow$ benign.

2.4.6 Giai đoạn 6, Sinh báo cáo

ReportGenerator tổng hợp toàn bộ kết quả thành bộ tài liệu case đầy đủ, lưu trong thư mục `CASE_{OS}_{timestamp}/`. Threat Score được tính từ 0–100 dựa trên số lượng IOC theo mức độ và số kỹ thuật MITRE phát hiện được, phân loại thành bốn mức: LOW (0–39), MEDIUM (40–59), HIGH (60–79), CRITICAL (80–100). File report.md sinh bằng Markdown có hyperlink trực tiếp đến từng technique tại attack.mitre.org. Khi không có API key nào được cấu hình, hệ thống tự động chèn cảnh báo vào đầu báo cáo để người đọc biết kết quả chỉ dựa trên whitelist cục bộ.

2.5 Công nghệ và giải pháp

2.5.1 Volatility3, Framework phân tích bộ nhớ

Volatility3 là framework mã nguồn mở được viết lại hoàn toàn bằng Python 3, ra mắt sau Volatility2 với nhiều cải tiến kiến trúc đáng kể (Volatility Foundation, n.d.). Điểm khác biệt quan trọng nhất là cơ chế symbol-based analysis: thay vì dựa vào cấu trúc KDBG cố định, Volatility3 sử dụng Intermediate Symbol Format (ISF), bảng ký hiệu debug sinh ra từ PDB của từng phiên bản OS, để phân tích chính xác cấu trúc bộ nhớ kernel (Pen Test Partners, 2025). Điều này cho phép framework hỗ trợ ổn định Windows 10/11, kể cả các bản cập nhật mới nhất, trong khi Volatility2 gặp khó khăn với các hệ điều hành hiện đại. Ngoài ra, Volatility3 hỗ trợ đầu ra JSON native phù hợp trực tiếp với kiến trúc pipeline tự động.

Trong hệ thống, VolatilityExecutor gọi Volatility3 qua `asyncio.create_subprocess_exec` với flag `-r json`, nhận output từ stdout và parse trực tiếp thành Python object để xử lý tiếp. Hệ thống tự dò lệnh Volatility3 theo thứ tự ưu tiên: `file vol.py standalone` $\rightarrow$ `module python3 -m vol.py` $\rightarrow$ lệnh `vol` trong PATH, đảm bảo hoạt động trong nhiều môi trường triển khai khác nhau.

Volatility Foundation. (n.d.). *volatilityfoundation/volatility3: Volatility 3.0 development*.

2.5.2 Model Context Protocol (MCP), Giao thức kết nối AI Agent

Model Context Protocol (MCP) là chuẩn giao thức mở do Anthropic công bố vào tháng 11 năm 2024, với mục tiêu giải quyết bài toán tích hợp tổ hợp $M \times N$, khi có M mô hình AI và N công cụ khác nhau, mỗi cặp tích hợp yêu cầu một connector tùy biến riêng biệt (Anthropic, 2024). MCP đưa ra giao diện thống nhất dựa trên JSON-RPC 2.0, trong đó AI Host kết nối đến MCP Server thông qua MCP Client; MCP Server đăng ký các *tool*, *resource* và *prompt template* mà AI có thể tự khám phá và gọi theo ngữ cảnh hội thoại (Wikipedia, 2025).

So với REST API truyền thống, MCP có ba lợi thế rõ ràng trong ngữ cảnh agentic: (1) AI tự khám phá danh sách tool khả dụng tại runtime thay vì cần biết schema cứng

trước; (2) AI quyết định thứ tự và cách kết hợp các tool dựa trên kết quả trung gian; (3) giao thức hỗ trợ stateful session phù hợp với phân tích nhiều bước. Kể từ khi ra mắt, MCP đã được OpenAI và Google DeepMind chấp nhận áp dụng (Wikipedia, 2025).

Anthropic. (2024, November 18). *Introducing the Model Context Protocol*.

2.5.3 VirusTotal và AbuseIPDB, Threat Intelligence

VirusTotal là nền tảng threat intelligence của Google cho phép tra cứu địa chỉ IP, tên miền và giá trị băm file thông qua REST API. Mỗi đối tượng được phân tích bởi hơn 70 engine antivirus và sandbox, kết quả trả về dưới dạng last_analysis_stats gồm số lượng engine phát hiện malicious, suspicious, harmless và undetected (Google Threat Intelligence, n.d.). Trong hệ thống, VirusTotalValidator tính điểm theo công thức $(\text{malicious} + \text{suspicious} \times 0.5) / \text{total}$, với ngưỡng cảnh báo 0.30 cho IP/domain và 0.10 cho hash, ngưỡng hash thấp hơn do bất kỳ engine nào phát hiện hash độc hại đều là tín hiệu rất đáng tin cậy.

AbuseIPDB là cơ sở dữ liệu cộng đồng chuyên về danh tiếng địa chỉ IP, nơi quản trị viên hệ thống trên toàn cầu báo cáo các IP tham gia vào tấn công brute-force, spam, DDoS và các hành vi độc hại khác (AbuseIPDB, n.d.). API trả về abuseConfidencePercentage, tỷ lệ phần trăm tin cậy rằng IP đó đang có hành vi lạm dụng, tính dựa trên tần suất và độ gần đây của các báo cáo trong vòng 90 ngày gần nhất (isMalicious, 2026). Ngưỡng phán quyết trong hệ thống được đặt tại 50%.

Cả hai dịch vụ đều được bảo vệ bởi bộ nhớ đệm Redis TTL 6 giờ kết hợp với bộ nhớ đệm trong tiến trình để giảm thiểu số lần gọi API. Riêng VirusTotalValidator áp dụng thêm cơ chế kiểm soát tốc độ 15 giây giữa các request, đảm bảo tuân thủ giới hạn của tài khoản miễn phí (4 request/phút).

2.5.4 Redis, Bộ nhớ đệm phân tán

Redis (Remote Dictionary Server) là cơ sở dữ liệu lưu trữ trong bộ nhớ (in-memory), hỗ trợ nhiều kiểu dữ liệu và cơ chế TTL linh hoạt, được sử dụng rộng rãi trong các hệ thống yêu cầu độ trễ thấp (Redis, n.d.). Trong hệ thống, Redis phục vụ hai mục đích chính: lưu cache kết quả plugin Volatility3 (TTL 24 giờ) và cache kết quả tra cứu threat intelligence (TTL 6 giờ). Truy cập Redis được thực hiện qua thư viện redis.asyncio, cho phép các thao tác cache không chặn (non-blocking) trong vòng lặp sự kiện asyncio, đảm bảo pipeline không bị gián đoạn khi chờ phản hồi.

Mục đích	Khóa cache	TTL
Kết quả plugin Volatility3	vol3:{hash_dump}:{plugin}:{hash_args}	24 giờ
Kết quả VirusTotal	vt:{endpoint}	6 giờ

Kết quả AbuseIPDB	abuse:{ip}	6 giờ
-------------------	------------	-------

2.5.5 Docker Compose, Triển khai và đóng gói

Hệ thống được đóng gói bằng Docker để đảm bảo môi trường chạy đồng nhất, bao gồm Python, Volatility3, symbol tables và Redis trong cùng một stack. docker-compose.yml định nghĩa hai service: mcp-server (chạy MCP Server với Volatility3) và redis (cache). entrypt.sh thực hiện khởi tạo môi trường trước khi khởi động server, kiểm tra kết nối Redis, xác minh Volatility3 khả dụng, tạo thư mục cần thiết. healthcheck.sh được chạy định kỳ để đảm bảo service luôn sẵn sàng phục vụ. Toàn bộ cấu hình nhạy cảm (API keys, đường dẫn, timeout) được truyền qua biến môi trường, không hardcode trong mã nguồn.

2.6 Tổng kết

Chương 2 đã trình bày đầy đủ hành trình từ yêu cầu đến giải pháp của hệ thống. Xuất phát từ bài toán tự động hóa trích xuất IOC từ file dump RAM, nhóm đề xuất kiến trúc Automatic Volatility3 Pipeline IOC Extraction with AI Agent, một pipeline sáu giai đoạn tích hợp liền mạch từ thực thi plugin đến sinh báo cáo pháp y, được bọc bởi MCP Server để kết nối với AI Agent.

Điểm mạnh của hướng tiếp cận này nằm ở ba khía cạnh: (1) tự động hóa toàn diện, AI Agent điều phối toàn bộ quy trình từ một câu lệnh ngôn ngữ tự nhiên; (2) độ chính xác cao, cơ chế xác minh đa tầng kết hợp whitelist, VirusTotal, AbuseIPDB và Correlation Guard giảm thiểu dương tính giả; (3) hiệu quả tài nguyên, bộ nhớ đệm Redis hai tầng giúp tái sử dụng kết quả mà không cần tính toán lại.

Yêu cầu	Giải pháp	Công nghệ
Tự động thực thi plugin	Pipeline bất đồng bộ	Volatility3 + asyncio
Kết nối với AI Agent	Giao thức chuẩn	MCP Server
Trích xuất IOC đa loại	Ba tầng song song	Regex + Context-Aware + Registry Rules
Xác minh IOC	Đa tầng có trọng số	VirusTotal + AbuseIPDB + Whitelist
Giảm tải API & plugin	Cache hai tầng	Redis + in-memory
Môi trường đồng nhất	Containerization	Docker Compose

Chương 3 : Kiến trúc nền tảng

3.1 Phân tích bộ nhớ tiến trình Windows

3.1.1 VAD tree và quyền bảo vệ trang nhớ

Mỗi tiến trình Windows được cấp một không gian địa chỉ ảo riêng biệt. Để quản lý toàn bộ các vùng đã cấp phát trong không gian đó, Windows kernel duy trì một cấu trúc dữ liệu gọi là VAD tree (Virtual Address Descriptor tree), một cây tìm kiếm nhị phân tự cân bằng, mỗi node mô tả một vùng địa chỉ ảo liên tục với các thuộc tính: địa chỉ bắt đầu, địa chỉ kết thúc, quyền bảo vệ trang nhớ (page protection), và trạng thái ánh xạ (Ligh et al., 2014).

Quyền bảo vệ trang nhớ là thuộc tính quan trọng nhất từ góc độ pháp y. Windows định nghĩa các mức quyền theo hằng số:

Hằng số	Ý nghĩa	Mức độ nghi ngờ
PAGE_READONLY	Chỉ đọc	Bình thường
PAGE_READWRITE	Đọc + ghi	Bình thường (heap, stack)
PAGE_EXECUTE_READ	Đọc + thực thi	Bình thường (code section)
PAGE_EXECUTE_READWRITE	Đọc + ghi + thực thi	Rất đáng ngờ

PAGE_EXECUTE_READWRITE (RWX) là tổ hợp bất thường, một vùng nhớ hợp lệ thường *hoặc* có thể ghi (dữ liệu), *hoặc* có thể thực thi (code), không bao giờ cần cả hai cùng lúc. Khi kẻ tấn công inject shellcode theo kỹ thuật VirtualAllocEx + WriteProcessMemory, họ buộc phải cấp quyền RWX để vừa ghi code vào vùng nhớ vừa thực thi nó (Ligh et al., 2014).

Tuy nhiên, chỉ có RWX chưa đủ để khẳng định, một số thư viện JIT compiler hợp lệ (Java VM, .NET CLR) cũng dùng RWX. Điều kiện bổ sung quyết định là tiêu đề PE (MZ, hai byte đầu 0x4D 0x5A): nếu một vùng nhớ RWX bắt đầu bằng MZ, điều đó có nghĩa là một file thực thi (.exe hoặc .dll) đã được nạp thủ công vào vùng nhớ đó mà không qua cơ chế nạp module chuẩn của Windows. Đây là đặc trưng của Reflective DLL Injection và các biến thể shellcode loader hiện đại.

Trong ContextAwareExtractor, phương thức analyze_malfind() triển khai logic này trực tiếp từ output của plugin windows.malware.malfind.Malfind:

```
_rwx = re.compile(r"PAGE_EXECUTE_READ(WRITE)?|rwx", re.IGNORECASE)

for entry in malfind_data:
```

```

protection = str(_get(entry, "Protection", "protection", "Protect"))
if not _rwx.search(protection):
    continue

hexdump = str(_get(entry, "Hexdump", "hexdump", "HexDump",
"Disasm", "disasm"))
has_mz = bool(re.search(r"^(MZ|4D\s*5A)", hexdump[:40],
re.IGNORECASE))
pid = _get(entry, "PID", "pid", "Pid")
process = str(_get(entry, "Process", "name", "Name"))
start_vpn = str(_get(entry, "Start VPN", "StartVPN", "start",
"VadStart"))

confidence = 0.90 if has_mz else 0.70

```

Điểm tin cậy 0.90 khi có cả RWX lẫn MZ, và 0.70 khi chỉ có RWX, phản ánh chính xác mức độ chắc chắn khác nhau giữa hai trường hợp. Kỹ thuật MITRE được gán cứng là T1055 (Process Injection). Với plugin windows.malware.hollowprocesses.HollowProcesses, phương thức analyze_hollow_processes() xử lý riêng kỹ thuật Process Hollowing (T1055.012), trường hợp tiến trình có vỏ hợp lệ nhưng nội dung đã bị thay thế, với điểm tin cậy 0.88.

3.1.2 EPROCESS linked list và pool scanning

Windows kernel quản lý danh sách tiến trình đang chạy thông qua một doubly-linked list (danh sách liên kết vòng đôi) với điểm neo là biến kernel PsActiveProcessHead. Mỗi _EPROCESS, cấu trúc kernel đại diện cho một tiến trình, chứa trường ActiveProcessLinks là một node trong danh sách này, trỏ đến _EPROCESS của tiến trình trước và tiến trình kế tiếp (Ligh et al., 2014).

Plugin windows.pslist.PsList duyệt danh sách này từ PsActiveProcessHead và liệt kê tất cả tiến trình tìm được. Kỹ thuật phòng thủ DKOM (Direct Kernel Object Manipulation) của rootkit cắt tiến trình ra khỏi linked list bằng cách sửa con trỏ Flink/Blink của hai node lân cận để bỏ qua node mục tiêu, tiến trình biến mất khỏi pslist nhưng vùng nhớ _EPROCESS vẫn còn nguyên trong RAM (Ligh et al., 2014).

Pool scanning là phương pháp thay thế không phụ thuộc vào linked list. Windows kernel cấp phát bộ nhớ cho các đối tượng kernel (bao gồm _EPROCESS) qua Pool Allocator, và mỗi block được cấp phát có một pool tag, chuỗi bốn byte định danh loại đối tượng. Pool tag của _EPROCESS là Proc. Plugin windows.psscan.PsScan quét toàn bộ bộ nhớ vật lý tìm tất cả các block có tag Proc, sau đó xác nhận bằng các trường nội bộ như magic number và offset hợp lệ, hoàn toàn độc lập với linked list.

Phương thức `analyze_hidden_processes()` trong `ContextAwareExtractor` vận dụng sự khác biệt này bằng cách diff kết quả của hai plugin:

```
if str(pid) in pslist_pids:
    continue # visible in pslist, normal

# Terminated process still in physical memory, NOT rootkit-hidden
if exit_time and exit_time not in ("", "None", "0", "N/A",
                                    "1970-01-01 00:00:00",
                                    "1970-01-01T00:00:00"):

    continue

iocs.append(IOC(
    ioc_type=IOCType.PROCESS,
    value=f"{process} (PID {pid})",
    confidence=0.92,
    source_plugin="windows.psscan.PsScan",
    context={
        "pid": pid,
        "process": process,
        "offset": offset,
        "technique": "T1564.001",
        "details": "Process in psscan but not pslist, likely
DKOM-hidden by rootkit",
        "category": "host",
    },
    extracted_at=datetime.now(),
))
```

Logic lọc `ExitTime` là điểm quan trọng: tiến trình đã kết thúc vẫn có thể còn `_EPROCESS` trong RAM do pool chưa được giải phóng, và sẽ xuất hiện trong `psscan` nhưng vắng mặt trong `pslist`, đây không phải dấu hiệu rootkit. Việc kiểm tra `ExitTime` loại bỏ các false positive này trước khi gán điểm tin cậy 0.92 cho kết quả còn lại.

3.2 Symbol Table và ISF trong Volatility3

Vấn đề cần giải quyết

Để đọc thông tin từ một file dump RAM, công cụ phân tích cần biết chính xác tại địa chỉ offset nào trong cấu trúc `_EPROCESS` thì lưu trường `ImageFileName`, offset nào là `UniqueProcessId`, offset nào là `ActiveProcessLinks`. Những giá trị này không cố định, chúng thay đổi giữa từng phiên bản Windows, từng bản cập nhật, thậm chí từng hotfix nhỏ. Volatility2 giải quyết bằng cách đóng gói thông tin này thành profile, file cấu hình

cứng cho từng phiên bản OS, phải chỉ định thủ công (--profile Win10x64_19041). Đây là điểm yếu lớn: mỗi bản vá mới đều cần profile mới, và khi không có profile chính xác thì toàn bộ plugin sai kết quả.

Giải pháp của Volatility3 : ISF từ PDB

Volatility3 giải quyết bằng cơ chế Intermediate Symbol Format (ISF), file JSON mô tả đầy đủ layout bộ nhớ của kernel cho một phiên bản OS cụ thể: tên kiểu dữ liệu, kích thước, offset từng trường, giá trị enum, và địa chỉ của các kernel symbol quan trọng như PsActiveProcessHead (Volatility Foundation, n.d.).

ISF được sinh ra từ file PDB (Program Database), file debug symbol mà Microsoft phân phối công khai tại Symbol Server cho tất cả bản Windows đã phát hành. Mỗi build Windows có một PDB riêng, và mỗi PDB sinh ra một ISF riêng. Khi gặp dump từ một bản Windows chưa có ISF trong cache, Volatility3 tự tải PDB từ Microsoft Symbol Server và chuyển đổi thành ISF, không cần người dùng can thiệp.

Trong hệ thống, biến VOLATILITY_SYMBOL_PATH trong Dockerfile trỏ đến thư mục ISF đã được tải sẵn vào image. Dòng sau trong VolatilityExecutor.run_plugin() truyền đường dẫn này khi gọi Volatility3:

```
if self.symbol_dirs and Path(self.symbol_dirs).exists():
    cmd.extend(["-s", self.symbol_dirs])
```

Nhờ đó, toàn bộ plugin chạy trong container không cần kết nối mạng để tải symbol và không phụ thuộc vào cấu hình môi trường bên ngoài.

Cơ chế tự nhận diện OS

Điều kiện để chạy đúng plugin là biết trước dump thuộc Windows hay Linux, vì cùng một mục đích phân tích tiến trình nhưng Windows dùng windows.pslist.PsList trong khi Linux dùng linux.pslist.PsList, với ISF và cấu trúc kernel hoàn toàn khác nhau.

Hệ thống giải quyết bằng công cụ detect_os, bước bắt buộc đầu tiên trong pipeline, được ghi rõ trong instruction của MCP Server: *"NEVER skip"*. Hàm detect_os_helper() thực hiện theo thứ tự:

```
async def _detect_os_helper(dump_path: str) -> dict:
    try:
        result = await executor.run_plugin(dump_path,
            "windows.info.Info", renderer="json")
        if result.success and result.data:
            row = result.data[0]
            return {
                "os_type": "windows",
```

```

        "version": str(row.get("NtMajorVersion", "unknown")),
        "build": str(row.get("NtBuildNumber", "unknown")),
        "arch": "x64" if row.get("Is64Bit", True) else "x86",
    }
except Exception:
    pass
try:
    result = await executor.run_plugin(dump_path, "banners.Banners",
renderer="json")
    if result.success and result.data:
        for banner in result.data:
            match = re.search(r"Linux version (\d+\.\d+\.\d+)",
banner.get("Banner", ""))
            if match:
                return {"os_type": "linux", "version":
match.group(1), "build": None, "arch": "x64"}
except Exception:
    pass
return {"os_type": "windows", "version": "unknown", "build": "unknown",
"arch": "x64"}

```

windows.info.Info hoạt động bằng cách đọc trực tiếp cấu trúc `_KUSER_SHARED_DATA` trong kernel, một vùng nhớ đặc biệt luôn có địa chỉ cố định (0x7FFE0000 trong user space, được mirror vào 0xFFFF0000 trong kernel space) trên mọi phiên bản Windows, chứa `NtMajorVersion`, `NtBuildNumber`, và flag `Is64Bit`. Nếu dump không phải Windows thì plugin này thất bại hoàn toàn chứ không trả về kết quả sai, đây là lý do nó được dùng làm bước thăm dò đầu tiên. Kết quả `detect_os` được cache vào file `osprofilecache.json` để các lần gọi sau trên cùng dump không phải chạy lại plugin.

Khi đã có OS type, DecisionEngine dùng thông tin này để chọn đúng bộ plugin:

```

profile = self.profiles[goal][os_type]
return TriagePlan(
    goal=goal,
    os_type=os_type,
    plugins=profile["plugins"],
    estimated_minutes=profile.get("estimated_minutes", 10),
    description=profile.get("description", ""),
)

```


Không có bước nào trong pipeline yêu cầu người dùng nhập tên profile thủ công, đây là điểm khác biệt căn bản so với quy trình Volatility2 truyền thống mà hệ thống được xây dựng để thay thế.

3.3 MITRE ATT&CK và thiết kế quy tắc phát hiện

Vấn đề cần giải quyết

Khi RegistryAnalyzer phát hiện giá trị bất thường trong khóa HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon, câu hỏi đặt ra là: bất thường này thuộc loại hành vi tấn công nào, nguy hiểm đến mức nào, và người phân tích cần làm gì tiếp theo? Nếu chỉ trả về kết quả "registry key đáng ngờ" mà không có ngữ cảnh, báo cáo không có giá trị thực tế. Nhóm cần một ngôn ngữ chung để phân loại hành vi, đó là lý do MITRE ATT&CK được đưa vào làm nền tảng thiết kế quy tắc, không phải như một tính năng bổ sung mà là khung phân loại từ đầu.

Cấu trúc MITRE ATT&CK

MITRE ATT&CK (Adversarial Tactics, Techniques & Common Knowledge) là cơ sở kiến thức mô tả hành vi kẻ tấn công theo ba cấp:

- **Tactic:** Mục tiêu tại một giai đoạn tấn công, *Persistence, Defense Evasion, Credential Access, Execution...*
- **Technique:** Cách thực hiện cụ thể để đạt mục tiêu, ví dụ T1547.001 *Registry Run Keys* thuộc tactic *Persistence*
- **Sub-technique:** Biến thể cụ thể, T1055.012 *Process Hollowing* là một cách thực hiện của T1055 *Process Injection*

Điều quan trọng là mỗi technique trên attack.mitre.org đi kèm phương pháp phát hiện và biện pháp phòng thủ được cộng đồng kiểm chứng, đây là thông tin nhóm khai thác trong phần khuyến nghị của báo cáo.

Thiết kế 18 quy tắc trong RegistryAnalyzer

REGISTRY_RULES là danh sách 18 quy tắc tĩnh, mỗi quy tắc có cấu trúc ba tầng khớp tuần tự:

key_pattern → bắt buộc, regex khớp với đường dẫn registry key

value_pattern → tùy chọn, regex khớp với tên value

data_patterns → tùy chọn, danh sách regex, khớp được bất kỳ một thì đủ

Quy tắc chỉ kích hoạt khi đi qua đủ các tầng không None. Nếu data_patterns được đặt là None, tức là sự hiện diện của key/value đã đủ để kích hoạt mà không cần kiểm tra dữ liệu. Ví dụ quy tắc phát hiện AppInit_DLLs:

```
{  
  "key_pattern": r"\\AppInit_DLLs$",  
  "value_pattern": None,  
}
```

```

    "data_patterns": [r".+"],
    "category": "persistence",
    "mitre": "T1546.010",
    "severity": "critical",
    "description": "AppInit_DLLs set,DLL loaded into every GUI process",
    "confidence": 0.90,
    "malware_families": ["dll_hijack"],
}

```

Các technique MITRE được ánh xạ trực tiếp trong từng quy tắc, phân theo bốn nhóm:

Category	Technique IDs	Ví dụ phát hiện
persistence	T1547.001, T1547.004, T1546.010, T1546.012, T1543.003, T1053.005	Run key, Winlogon hijack, AppInit_DLLs, IFEO Debugger, malicious service
defense_evasion	T1548.002, T1562.001, T1112	UAC disabled, Defender disabled/exclusion, Windows Update disabled
credential_access	T1547.005, T1003.001, T1556.002	Unknown LSA package, WDigest plaintext caching, custom credential provider
execution	T1574.007	System PATH pointing to suspicious directory

Mỗi quy tắc có thêm trường severity với bốn mức: critical, high, medium, low, ánh xạ ra số nguyên qua SEVERITY_RANK = {critical: 4, high: 3, medium: 2, low: 1}. Kết quả findings được sắp xếp giảm dần theo severity trước khi trả về, đảm bảo người phân tích thấy cảnh báo nguy hiểm nhất lên đầu. Confidence được gán tĩnh theo từng quy tắc, quy tắc WDigest plaintext credential caching có confidence 0.95 vì UseLogonCredential = 1 gần như không bao giờ tồn tại trong môi trường lành mạnh, trong khi quy tắc Scheduled task in registry chỉ có 0.60 vì đây là tính năng hợp lệ thường dùng.

Cơ chế lọc false positive

Có hai bộ lọc chạy trước khi áp dụng bất kỳ quy tắc nào:

- looks_like_live_registry_key(): loại bỏ các chuỗi trông giống đường dẫn file (.dat, .hve, .log, bắt đầu bằng \), đây là artifact từ các plugin như hivelist liệt kê file hive trên đĩa, không phải giá trị registry đang hoạt động trong bộ nhớ.

- `is_common_benign_autorun()`: loại bỏ các entry Run key hợp lệ phổ biến như OneDrive, Microsoft Teams, Security Health, những entry này luôn có mặt trong mọi máy Windows sạch và không cần báo cáo.

Riêng với Run/RunOnce key (T1547.001), có thêm kiểm tra authoritative hive: chỉ xét entry từ HKLM hoặc HKCU, bỏ qua nếu đến từ hive không xác định, tránh false positive từ backup hive hay hive offline được mount vào dump.

MITRE mapping trong báo cáo cuối

Phần MITRE ATT&CK của báo cáo không chỉ liệt kê technique ID mà còn nhóm theo tactic, đếm số IOC thuộc mỗi technique, và sinh hyperlink trực tiếp:

```
techniques = report.mitre.get("techniques", [])
if techniques:
    md += "---\n\n## 🚩 MITRE ATT&CK Coverage\n\n"
    md += "| ID | Technique | Tactic | IOC Count | Top Recommendation |\n"
    md += "|----|-----|-----|-----|-----|\n"
    for tech in techniques:
        if isinstance(tech, dict):
            rec = tech.get('recommendations', ['N/A'])[0] if
            tech.get('recommendations') else 'N/A'
            tid = tech.get('id', '')
            url =
            f"https://attack.mitre.org/techniques/{tid.replace('.', '/')}"
            md += (f"| [{tid}]({url}) "
                f"| {tech.get('name', 'N/A')} "
                f"| {tech.get('tactic', 'N/A')} "
                f"| {tech.get('ioc_count', 0)} "
                f"| {rec[:60]} |\n")
        md += "\n"
```

Hàm `calculate_threat_level()` dùng danh sách critical techniques, T1055, T1059.001, T1105, T1071, để cộng bonus điểm vào threat score:

```
critical_techniques = {"T1055", "T1059.001", "T1105", "T1071"}
critical_count = sum(1 for t in techniques if t.get("id") in
critical_techniques)
critical_bonus = critical_count * 5
```

Bốn technique này được chọn vì chúng là indicator của các giai đoạn nguy hiểm nhất trong một cuộc tấn công thực tế: code execution trong bộ nhớ (T1055), PowerShell

abuse (T1059.001), file download từ C2 (T1105), và giao tiếp C2 (T1071). Một dump có đủ bốn technique này đồng thời hầu như chắc chắn đang trong trạng thái compromise tích cực.

3.4 Cơ chế scoring VirusTotal và AbuseIPDB

Vấn đề cần giải quyết

Sau khi trích xuất IOC từ dump RAM, hệ thống cần trả lời câu hỏi: IP này có thực sự là C2 không, hay chỉ là server hợp lệ bị nhận diện nhầm? ContextAwareExtractor chỉ nhìn thấy hành vi trong bộ nhớ, nó không có thông tin về lịch sử của địa chỉ IP đó ngoài thực tế. VirusTotal và AbuseIPDB là hai nguồn threat intelligence bên ngoài bổ sung chiều thông tin này. Nhưng kết quả từ hai API có định dạng và thang đo khác nhau, và không thể đơn giản cộng chúng lại, đây là lý do ValidationPipeline cần một công thức trọng số rõ ràng.

Cơ chế VirusTotal: last_analysis_stats

Khi gửi một đối tượng đến VirusTotal (IP, domain, hoặc hash), API trả về trường last_analysis_stats với bốn nhóm kết quả từ toàn bộ engine đã quét: malicious, suspicious, harmless, undetected. VirusTotalValidator tính điểm từ ba nhóm đầu:

$$VT_{score} = \frac{malicious + suspicious * 0.5}{totalengines}$$

Trọng số 0.5 cho suspicious phản ánh tính không chắc chắn, engine báo suspicious có thể đúng hoặc sai với xác suất gần bằng nhau, không thể xếp ngang với malicious. Ngưỡng phán quyết được phân biệt theo loại IOC:

Loại IOC	Ngưỡng is_malicious	Lý do
IP / Domain	score $\geq$ 0.3	Tỉ lệ false positive cao hơn, IP shared hosting hay CDN thường bị vài engine báo sai
Hash (MD5/SHA1/SHA256)	score $\geq$ 0.1	Bất kỳ engine nào nhận diện một hash là malicious đều là tín hiệu đáng tin, hash không thể "trùng hợp" như IP

```
async def check_hash(self, hash_value: str) -> ValidationResult:
```

```

return self._parse_stats(await self._request(f"files/{hash_value}"),
threshold=0.1)

```

Rate limit được xử lý bằng cách chờ cứng 15 giây giữa các request (`wait = max(0.0, 15.0 - (now - self.last_request))`), đảm bảo tuân thủ giới hạn 4 request/phút của API miễn phí mà không cần retry logic phức tạp.

Cơ chế AbuseIPDB: abuseConfidencePercentage

AbuseIPDB xây dựng trên mô hình crowdsourced reporting: quản trị viên hệ thống và nhà cung cấp dịch vụ bảo mật trên toàn thế giới báo cáo các IP tấn công họ. Trường `abuseConfidencePercentage` trong response là một số từ 0–100, được tính từ: số lần báo cáo, độ gần đây (báo cáo trong 90 ngày gần đây có trọng số cao hơn), và sự đa dạng nguồn báo cáo, cùng IP bị nhiều tổ chức khác nhau báo cáo sẽ có điểm cao hơn nhiều lần báo cáo từ cùng một nguồn (AbuseIPDB, n.d.).

AbuseIPDBValidator chuyển `abuseConfidencePercentage` về thang 0–1 và dùng ngưỡng 50% cho `is_malicious`:

```

confidence = abuse_data.get("abuseConfidencePercentage", 0) / 100
result = ValidationResult(
    source="abuseipdb",
    is_malicious=confidence > 0.5,
    score=confidence,
    reason=f"AbuseIPDB: {confidence*100:.0f}% confidence,
{total_reports} reports",
    raw_data={
        "confidence": confidence,
        "total_reports": total_reports,
        "country": abuse_data.get("countryCode"),
        "isp": abuse_data.get("isp"),
    },
)

```

Ngưỡng 50%, thay vì cao hơn, được chọn có chủ đích: IP C2 trong chiến dịch tấn công có chủ đích (APT) thường được thay đổi thường xuyên và có thể chưa tích lũy đủ báo cáo để đạt điểm cao. AbuseIPDB chỉ áp dụng cho IOC loại IP vì API không hỗ trợ domain hay hash, đây là lý do validator chỉ gọi `self.abuse` khi `ioc.ioc_type` in ("ip", "ipv4").

Công thức trọng số kết hợp

Điểm cuối cùng của một IOC được tính bằng trung bình có trọng số từ tất cả validator đã chạy. Trọng số mặc định được định nghĩa tại khởi tạo `ValidationPipeline`:

```
self.weights = {"virustotal": 0.4, "abuseipdb": 0.3, "whitelist": 0.3}
```

$$finalscore = \frac{\sum_i score_i * w_i}{\sum_i w_i}$$

Tuy nhiên có một ngoại lệ quan trọng: khi VirusTotal trả về điểm ≥ 0.40 , trọng số VT được nâng động lên 0.50 để phản ánh độ tin cậy cao hơn của đồng thuận đa engine:

```
if vt_result and vt_result.score >= 0.40:
    vt_weight = 0.50
    ab_weight = 0.15 if (self.abuse and ioc.ioc_type in ("ip", "ipv4"))
else 0.0
total_w = vt_weight + ab_weight

final_score = (
    vt_result.score * vt_weight
    + (results[-1].score * ab_weight if ab_weight else 0)
) / (total_w or 1)
```

Trường hợp VT không kết luận rõ (score < 0.40), công thức trung bình trọng số tiêu chuẩn được áp dụng, cho phép AbuseIPDB đóng góp tương đương.

Cache hai tầng và phán quyết cuối

Cả hai validator đều dùng cùng chiến lược cache: kiểm tra memory cache (dict Python trong tiến trình) trước, nếu miss thì kiểm tra Redis, nếu vẫn miss mới gọi API thực sự, sau đó ghi vào cả hai tầng. TTL là 6 giờ cho cả VirusTotal lẫn AbuseIPDB (CACHE_TTL = 21600). Lý do TTL 6 giờ thay vì 24 giờ: thông tin threat intelligence có thể thay đổi trong ngày, một IP bị thêm vào blacklist hoặc xóa khỏi blacklist đều có ý nghĩa, khác với kết quả plugin Volatility3 (dump là bất biến, cache 24 giờ hợp lý).

Phán quyết cuối dùng ba ngưỡng cứng:

```
def _determine_verdict(self, score: float) -> str:
    if score >= 0.7:
        return "malicious"
    elif score >= 0.4:
        return "suspicious"
    return "benign"
```

Sau khi tất cả IOC được phán quyết, Correlation Guard chạy một lần cuối: các IOC loại injection, process, command, tức là kết quả từ phân tích hành vi, không có xác nhận từ external API, bị hạ xuống suspicious và giới hạn confidence tối đa 0.69 nếu không có ít nhất một IOC loại "bằng chứng cứng" (ip, hash, domain) nào được xác nhận là malicious cùng lúc. Điều này đảm bảo một finding đơn lẻ không có corroboration không bao giờ tự mình kết luận malicious.

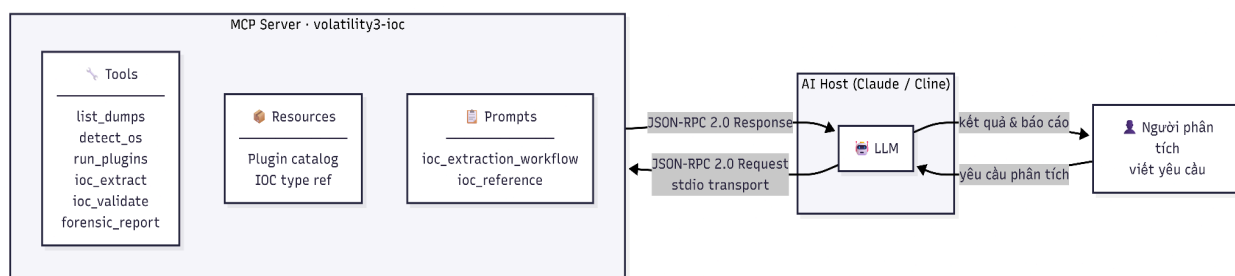
3.5 Model Context Protocol (MCP)

Vấn đề cần giải quyết

Pipeline phân tích dump RAM gồm nhiều bước tuần tự có phụ thuộc: phải có kết quả detect_os trước mới biết chạy plugin nào, phải có kết quả plugin trước mới trích xuất được IOC, phải có IOC trước mới validate được. Nếu đóng gói toàn bộ thành một REST API duy nhất, AI không thể can thiệp vào giữa để hỏi thêm hoặc điều chỉnh tham số. Nhóm cần một cơ chế cho phép AI gọi từng bước một cách có kiểm soát, nhận kết quả trung gian, và tự quyết định bước tiếp theo, đây là bài toán MCP giải quyết.

Cách MCP vận hành

Model Context Protocol (MCP) là giao thức chuẩn hóa do Anthropic công bố năm 2024, định nghĩa cách một LLM (AI Host) giao tiếp với các công cụ bên ngoài (MCP Server) thông qua JSON-RPC 2.0 (Anthropic, 2024). Kiến trúc gồm ba thành phần:



Khi khởi động, MCP Server gửi danh sách tool schemas cho AI Host, mỗi schema mô tả tên tool, mô tả chức năng, và các tham số đầu vào với kiểu dữ liệu. AI Host đọc schema này và tự biết khi nào nên gọi tool nào, với tham số gì, mà không cần code cứng logic điều phối. Giao tiếp qua stdio transport, server đọc JSON từ stdin và ghi JSON ra stdout, cho phép đóng gói toàn bộ trong Docker container mà không cần mở port mạng.

Ba primitive của MCP

MCP định nghĩa ba loại primitive mà server có thể cung cấp (Anthropic, 2024):

Primitive	Vai trò	Dùng trong hệ thống
-----------	---------	---------------------

Tool	Hành động có side effect, AI gọi chủ động	6 tool pipeline chính
Resource	Dữ liệu tĩnh AI có thể đọc	Danh sách plugin catalog, IOC type reference
Prompt	Template hướng dẫn workflow	ioc_extraction_workflow, ioc_reference

Tool là primitive quan trọng nhất trong hệ thống. Mỗi tool được đăng ký với decorator `@mcp.tool()`, kèm name và description, description không chỉ là tài liệu mà là instruction cho AI: AI đọc description để quyết định khi nào gọi tool đó.

Trường instructions trong FastMCP là system-level instruction được đưa vào context của AI ngay khi kết nối, đây là cơ chế để nhóm ràng buộc thứ tự thực thi 6 bước mà không cần AI tự suy luận.

Sử dụng tools hợp lý

Nếu gộp toàn bộ pipeline thành một tool duy nhất `analyze_dump()`, AI không thể kiểm soát từng bước, không thể dừng lại để báo cáo tiến độ, và không thể tái sử dụng kết quả trung gian. Việc chia thành 6 tool riêng biệt phục vụ ba mục đích thực tế:

- Kết quả trung gian có thể tái dùng: `run_plugins` lưu output vào file JSON và trả về `result_id`. Các bước sau dùng `result_id` thay vì truyền toàn bộ dữ liệu qua MCP, tránh vượt context window của LLM khi dump lớn sinh ra hàng nghìn dòng output.
- AI có thể điều chỉnh giữa chừng: Nếu `ioc_extract` trả về zero IOC, AI có thể gọi lại `run_plugins` với profile khác mà không phải chạy lại từ đầu.
- Người phân tích có thể quan sát từng bước: AI báo cáo kết quả của mỗi tool cho người dùng trước khi tiếp tục, tạo khả năng human-in-the-loop tự nhiên.

3.6 Caching strategy: TTL và two-level cache

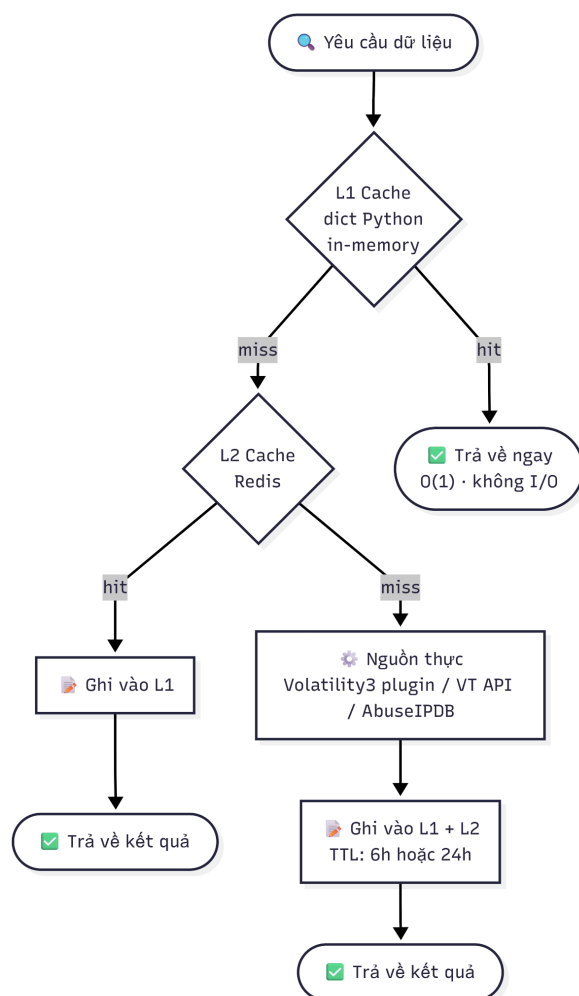
Vấn đề cần giải quyết

Hai thao tác tốn kém nhất trong pipeline là chạy plugin Volatility3 (mỗi plugin mất 30 giây đến 30 phút tùy dump size) và gọi API threat intelligence (bị giới hạn rate). Nếu người phân tích chạy lại pipeline trên cùng dump, hoặc hai IOC giống nhau xuất hiện ở hai plugin khác nhau, hệ thống không nên tốn thời gian lặp lại những tính toán đã có kết quả. Đây là lý do cache được thiết kế từ đầu, không phải thêm vào sau.

Một tầng cache duy nhất không đủ vì hai lý do đối lập:

- Redis một mình: Mỗi lần đọc cache phải qua network (dù là localhost), tạo overhead không cần thiết cho các giá trị được đọc lặp đi lặp lại trong cùng một lần chạy.

- Dict Python một mình: Mất toàn bộ khi tiến trình khởi động lại, và không chia sẻ được giữa nhiều worker.
- Two-level cache kết hợp ưu điểm của cả hai:



Trong VirusTotalValidator, L1 là self.mem_cache (dict), L2 là Redis. Trong VolatilityExecutor, L1 là self.dump_hashes (dict lưu hash của dump), L2 là Redis với key vol3:{dump_hash}:{plugin}:{args_hash}.

TTL theo đặc điểm của dữ liệu

TTL (Time-To-Live) được chọn dựa trên tần suất thay đổi thực tế của từng loại dữ liệu:

Loại dữ liệu	TTL	Lý do
Kết quả plugin Volatility3	24 giờ (86400s)	Dump RAM là bất biến, cùng dump, cùng plugin, cùng args sẽ luôn cho cùng kết quả

VirusTotal / AbuseIPDB	6 giờ (21600s)	Threat intelligence thay đổi trong ngày: IP có thể được thêm vào hoặc xóa khỏi blacklist
OS profile cache	File JSON không có TTL	Tái sử dụng trên nhiều phiên, OS của dump không bao giờ thay đổi

Cache key cho plugin được thiết kế đủ chi tiết để tránh collision:

```
dump_hash = await self.get_dump_hash(dump_path)
args_hash = hashlib.md5(
    json.dumps(args or {}, sort_keys=True).encode()
).hexdigest()[:8]
cache_key = f"vol3:{dump_hash}:{plugin}:{args_hash}"
```

Dùng SHA256 của 100MB đầu thay vì hash toàn bộ dump vì dump thực tế có thể lên đến 32GB, hash một phần giảm thời gian tính từ vài phút xuống dưới một giây, trong khi vẫn đủ phân biệt các dump khác nhau.

Semaphore(3): giới hạn song song có chủ đích

run_plugins_parallel() trong VolatilityExecutor dùng asyncio.Semaphore(3), tối đa 3 plugin chạy song song tại một thời điểm:

```
semaphore = asyncio.Semaphore(max_concurrent)
results = {}

async def run_with_semaphore(plugin_config: Dict[str, Any]) -> tuple:
    async with semaphore:
        name = plugin_config["name"]
        args = plugin_config.get("args") or {}
        result = await self.run_plugin(dump_path, name, args)
        return name, result

completed = await asyncio.gather(
    *[run_with_semaphore(p) for p in plugins],
    return_exceptions=True,
)
```

Con số 3 không phải tùy ý, đây là điểm cân bằng giữa hai ràng buộc thực tế. Mỗi plugin Volatility3 là một tiến trình con độc lập tốn khoảng 1–2 GB RAM để ánh xạ dump vào bộ nhớ. Với dump 8 GB trên máy 16 GB RAM, chạy 5 plugin song song có nguy cơ OOM; chạy 1 plugin tại một thời lúc lãng phí CPU khi các plugin đang chờ I/O đọc

dump. Con số 3 cho phép tận dụng I/O concurrency mà không gây memory pressure nguy hiểm. ValidationPipeline dùng Semaphore(5) cao hơn vì các API call nhẹ hơn nhiều, chỉ tốn network I/O, không tốn RAM.

Chương 4 : ỨNG DỤNG

4.1 Môi trường thực nghiệm

4.1.1 Máy ảo windows 11

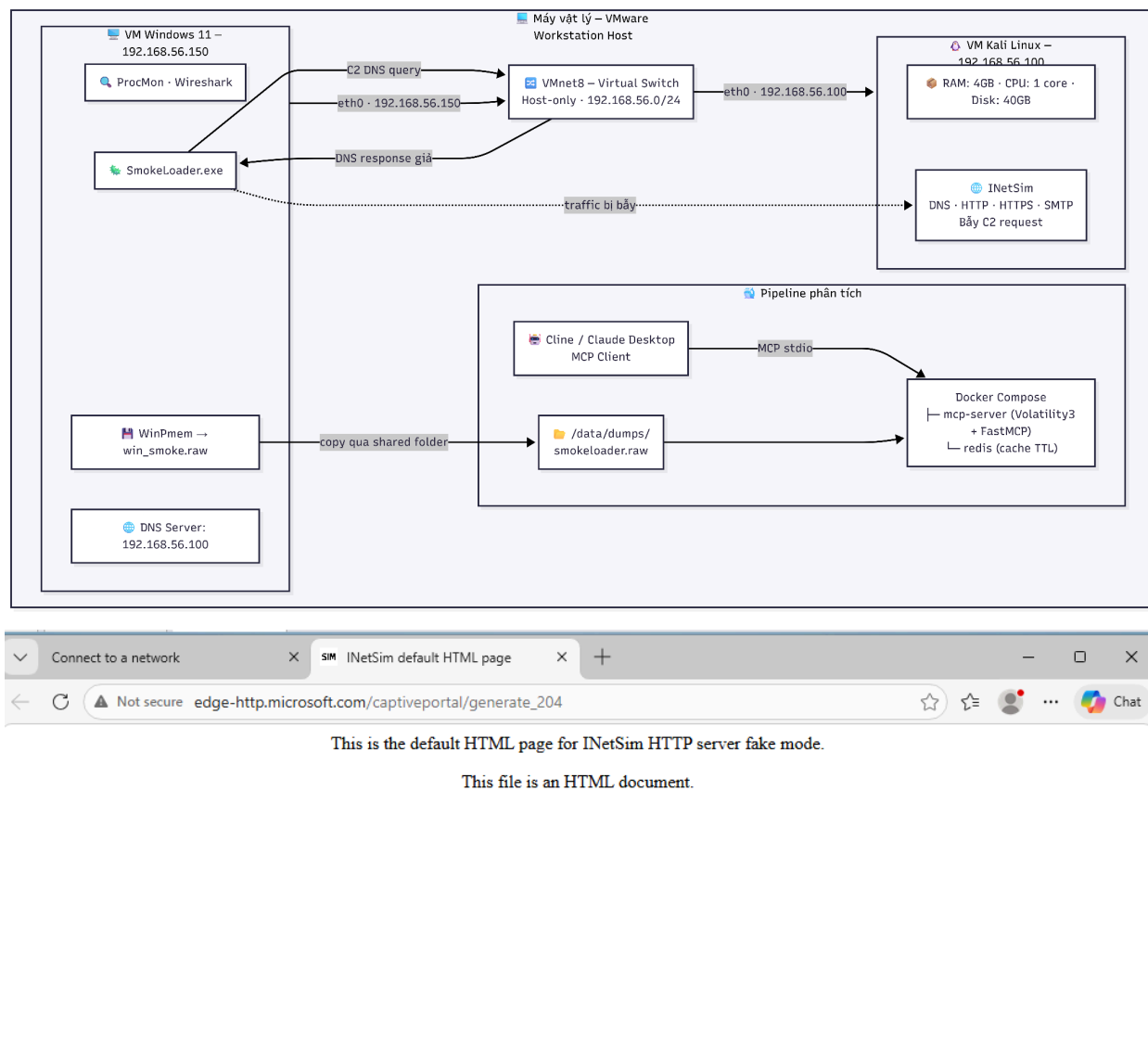
Máy ảo Windows 11 đóng vai trò nơi mẫu malware được thực thi có kiểm soát để quan sát hành vi. Toàn bộ cấu hình được tối giản hóa nhằm đảm bảo malware chạy đủ hành vi mà không bị phát hiện là môi trường ảo hóa.

Thông số cấu hình:

Thông số	Giá trị
Hệ điều hành	Windows 11 x64
RAM	4 GB
CPU	2 core
Ổ cứng hệ thống	50 GB NVMe
Ổ cứng lưu dump	10 GB NVMe (ổ E:\)
Network Adapter	Custom VMnet8,Host-only
IP	192.168.56.150
DNS Server	192.168.56.100 (trở về Kali)
Windows Defender	Tắt hoàn toàn

Các công cụ cài đặt sẵn trong VM:

Công cụ	Mục đích
Process Monitor (ProcMon)	Ghi lại toàn bộ hành vi process, thao tác file và registry trong thời gian thực
Wireshark	Capture traffic mạng trên interface VMnet8
WinPmem	Thu thập RAM dump ra file .raw trên ổ E:\



Hình : Kết nối INetsim

4.1.2 Máy ảo Kali linux

Máy ảo Kali Linux đóng vai trò giả lập hạ tầng mạng cho malware trong quá trình thực nghiệm. Kali không tham gia vào việc phân tích RAM dump, nhiệm vụ duy nhất của máy này là đảm bảo SmokeLoader chạy đủ hành vi bằng cách phản hồi mọi kết nối mạng mà malware tạo ra.

Thông số cấu hình:

Thông số	Giá trị
Hệ điều hành	Kali Linux 2024.x
RAM	4 GB

CPU	1 core
Ổ cứng	40 GB SCSI
Network Adapter	Custom VMnet8,Host-only
IP	192.168.56.100

INetSim, Internet Services Simulation:

INetSim là công cụ giả lập các dịch vụ internet phổ biến, lắng nghe trên địa chỉ 192.168.56.100 và phản hồi mọi request đến từ máy Windows mà không cần kết nối internet thật. Các dịch vụ được bật trong thực nghiệm.

```
Parsing configuration file.
Configuration file parsed successfully.
=== INetSim main process started (PID 2769) ===
Session ID: 2769
Listening on: 192.168.56.100
Real Date/Time: 2026-03-20 21:15:26
Fake Date/Time: 2026-03-20 21:15:26 (Delta: 0 seconds)
Forking services...
* dns_53_tcp_udp - started (PID 2771)
* irc_6667_tcp - started (PID 2781)
* finger_79_tcp - started (PID 2784)
* tftp_69_udp - started (PID 2780)
* syslog_514_udp - started (PID 2790)
* ntp_123_udp - started (PID 2783)
* time_37_tcp - started (PID 2792)
* ident_113_tcp - started (PID 2789)
* smtps_465_tcp - started (PID 2775)
* pop3_110_tcp - started (PID 2776)
* time_37_udp - started (PID 2793)
* smtp_25_tcp - started (PID 2774)
* https_443_tcp - started (PID 2773)
* ftp_21_tcp - started (PID 2778)
* daytime_13_tcp - started (PID 2794)
* http_80_tcp - started (PID 2772)
* pop3s_995_tcp - started (PID 2777)
* ftps_990_tcp - started (PID 2779)
* echo_7_tcp - started (PID 2796)
* daytime_13_udp - started (PID 2795)
* quotd_17_tcp - started (PID 2800)
* echo_7_udp - started (PID 2797)
* discard_9_tcp - started (PID 2798)
* quotd_17_udp - started (PID 2801)
* chargen_19_udp - started (PID 2803)
* discard_9_udp - started (PID 2799)
* chargen_19_tcp - started (PID 2802)
* dummy_1_udp - started (PID 2805)
* dummy_1_tcp - started (PID 2804)
done.
Simulation running.
```

Hình : INetsim hoạt động

4.1.3 Môi trường phân tích

Sau khi thu thập RAM dump từ máy Windows, toàn bộ quá trình phân tích được thực hiện trực tiếp trên máy vật lý host, tách biệt hoàn toàn với hai máy ảo detonation. Pipeline phân tích chạy trong Docker Compose gồm hai container: mcp-server chứa Volatility3 và FastMCP server, và redis phục vụ cache hai tầng. File RAM dump được mount vào container qua volume /data/dumps/. Người phân tích tương tác với pipeline thông qua Cline (VSCode extension) kết nối MCP Client đến mcp-server qua stdio transport, toàn bộ 6 bước pipeline được điều phối bằng ngôn ngữ tự nhiên thay vì gõ lệnh thủ công.

Việc tách pipeline ra host thay vì chạy trong Kali có hai lý do thực tế: host có tài nguyên CPU và RAM đầy đủ hơn để Volatility3 xử lý file dump 4 GB, và tránh ảnh hưởng đến INetSim đang chạy nền trong Kali trong suốt quá trình detonation.

4.2 Mẫu malware được chọn: SmokeLoader

Mẫu malware được chọn cho thực nghiệm là SmokeLoader, một họ malware thuộc dạng dropper/loader đã hoạt động từ năm 2011 và liên tục được cập nhật đến nay. Mẫu cụ thể sử dụng trong thực nghiệm có SHA256 7046e174e0f51284f119a0061ad816e066a5f5b24c775f8cfd613240c4a6912, được thu thập từ MalwareBazaar, ngày trang dưới tên file recuva_professional_patched.exe, giả mạo phần mềm khôi phục dữ liệu Recuva bản crack.

MalwareBazaar Database

You are currently viewing the MalwareBazaar entry for **SHA256 7046e174e0f51284f119a0061ad816e066a5f5b24c775f8cfd613240c4a6912**. While MalwareBazaar tries to identify whether the sample provided is malicious or not, there is no guarantee that a sample in MalwareBazaar is malicious.

Database Entry


Smoke Loader


Vendor detections: **13**

Intelligence **13**

IOCs

YARA **23**

File information

Comments

Actions

SHA256 hash:	 7046e174e0f51284f119a0061ad816e066a5f5b24c775f8cfd613240c4a6912
SHA3-384 hash:	 f62a5b571b331dad5645ea74e62ae2d36b1fb8b3e35e3c986b57135cb38e2e7db9622fd20084cb9194fed1e79b19b7df
SHA1 hash:	 98441a3afc50f2e9b88babeadd074d3cc8a7e2
MIME hash:	 881685404b0007c77c73e40f0f66e470

Hình : Mẫu SmokeLoader malware

SmokeLoader được chọn vì thể hiện đầy đủ các kỹ thuật tấn công qua bộ nhớ mà pipeline được thiết kế để phát hiện. Sau khi thực thi, SmokeLoader tiến hành process injection vào explorer.exe (T1055) rồi tự kết thúc tiến trình gốc, toàn bộ mã độc tiếp tục

chạy ẩn bên trong tiến trình hệ thống hợp lệ. Từ đó, nó drop và thực thi XMRig (miner tiền mã hoá) và một stealer module thu thập thông tin xác thực từ trình duyệt, ví tiền mã hoá và ứng dụng FTP. Cơ chế persistence được thiết lập qua registry Run key (T1547.001), và Defender bị vô hiệu hoá thông qua lệnh PowerShell mã hoá Base64 (T1059.001). Kết nối C2 được thực hiện đến các domain baxe.pics, coox.live qua các cổng không chuẩn như 48261 và 4219 (T1071).

Tổ hợp các kỹ thuật này bao phủ đồng thời cả hai nhóm IOC mà pipeline cần xác minh: network-based IOC (địa chỉ IP C2, domain, cổng kết nối bất thường) và host-based IOC (vùng nhớ RWX chứa shellcode, registry persistence, lệnh PowerShell độc hại, hash của payload được drop). Đây là lý do SmokeLoader phù hợp để đánh giá toàn diện khả năng phát hiện của hệ thống.

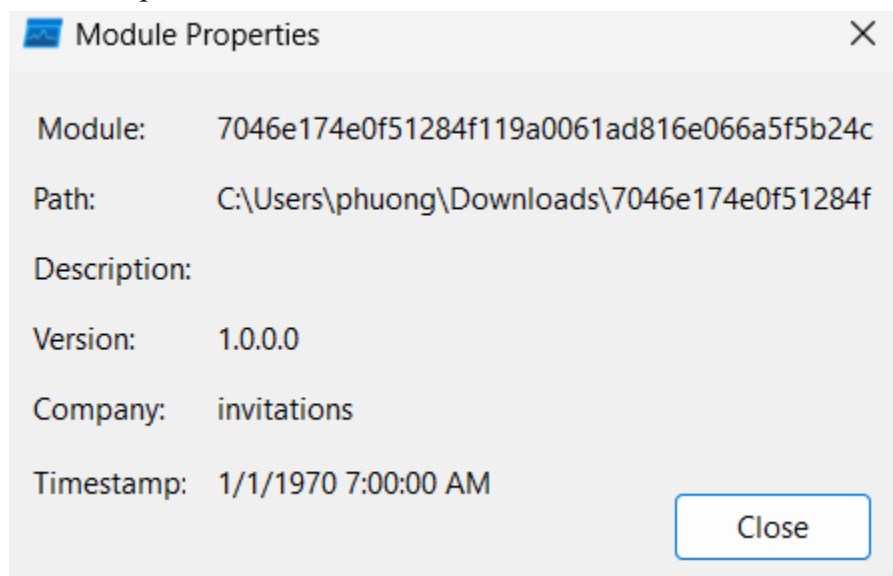
4.3 Thu thập bằng chứng hành vi

4.3.1 Process Monitor

Mẫu được thực thi với tên file là chuỗi SHA256 hash do tải trực tiếp từ MalwareBazaar. ProcMon ghi nhận tiến trình khởi chạy lúc 7:24:19 AM với PID 5460, Integrity Level High, dưới tài khoản DESKTOP-F7CQRH5\phuong. Đáng chú ý, tiến trình tự kết thúc lúc 7:24:49 AM, chỉ tồn tại 30 giây, đây là hành vi đặc trưng của SmokeLoader sau khi hoàn tất inject vào tiến trình hợp lệ của hệ thống.

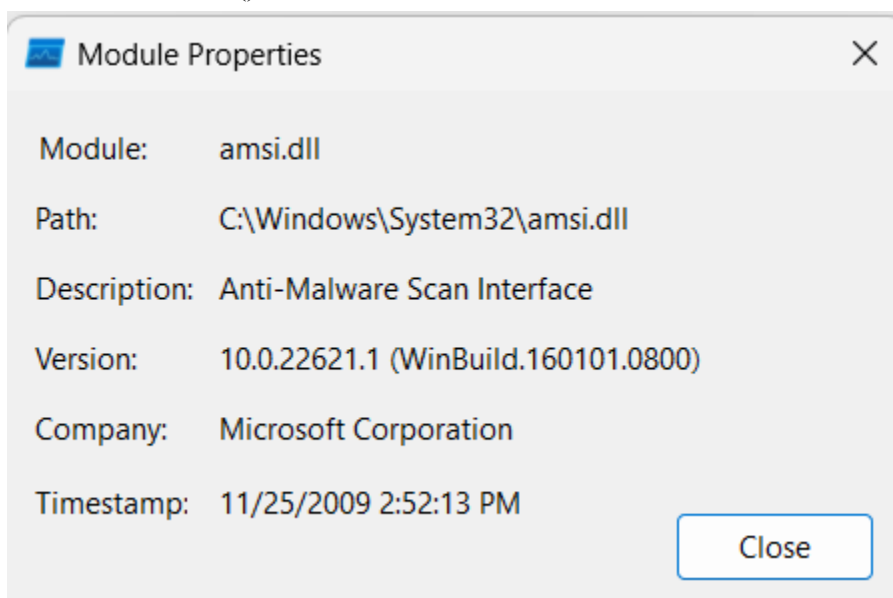
Các artifact ghi nhận được:

- Timestomping: Toàn bộ PE timestamp của file thực thi bị đặt về 1970-01-01 (Unix epoch), các DLL hệ thống có timestamp giả từ các năm bất thường (1904, 1912, 1913) hoặc tương lai (2027, 2036), đây là kỹ thuật anti-forensics nhằm gây khó khăn cho phân tích timeline.



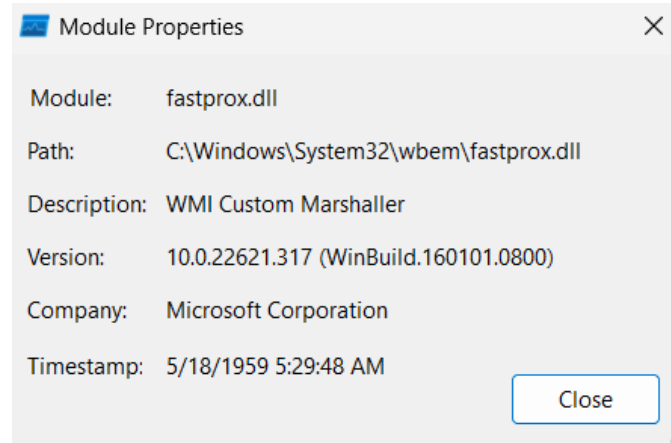
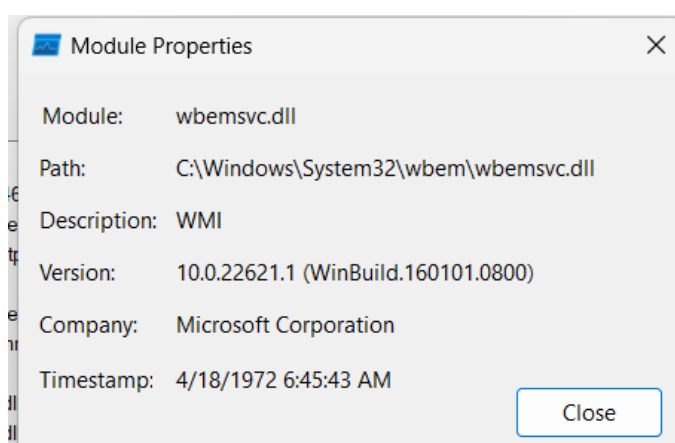
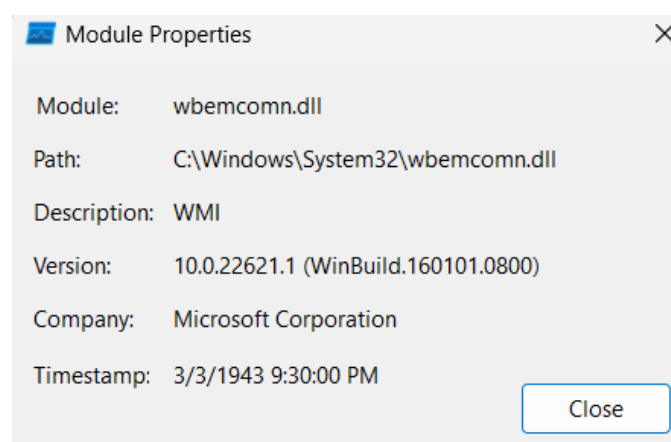
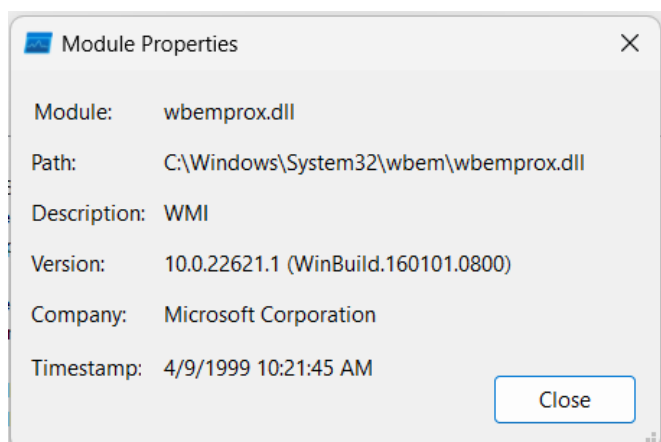
Hình : Timestomping

- AMSI Bypass: Malware load `amsi.dll`, cho thấy nỗ lực hook hoặc patch hàm `AmsiScanBuffer()` nhằm vô hiệu hóa Windows Defender scan.

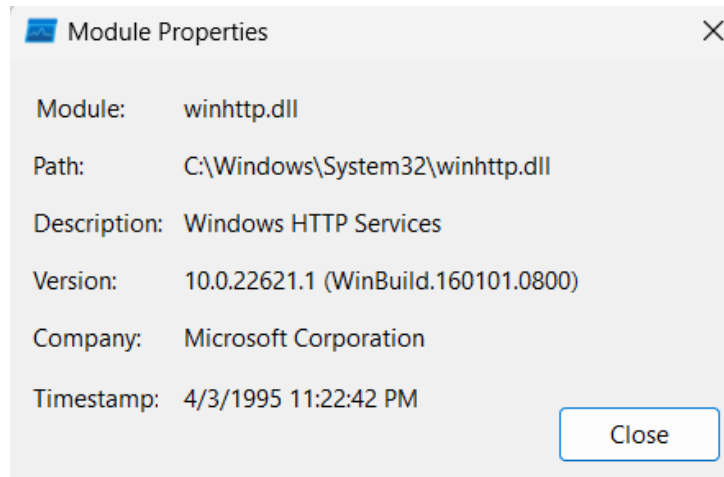


Hình : `amsi.dll`

- WMI Reconnaissance: Malware load 4 WMI DLL gồm `wbemsvc.dll`, `fastprox.dll`, `wbemprox.dll`, `wbemcomn.dll`, sử dụng WMI để query thông tin hệ thống.

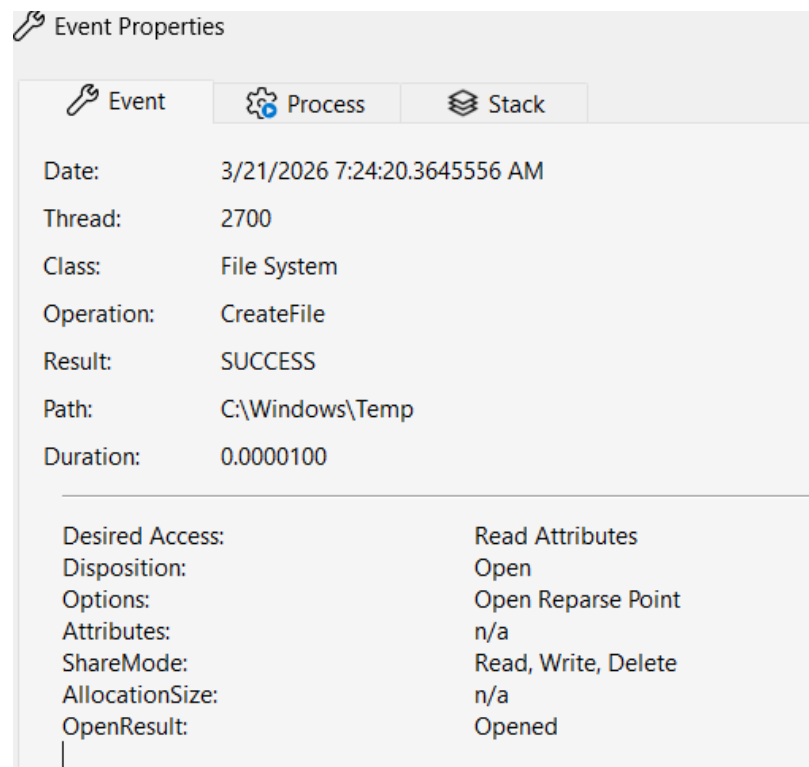


- HTTP Capability: Load winhttp.dll, chuẩn bị khả năng gửi HTTP request về C2 server.



- Temp Directory Access: Malware thực hiện CreateFile vào C:\Windows\Temp, hành vi reconnaissance thư mục trước khi inject.

7:24:20...	7046e174e0f512...	5460	CreateFile	C:\Windows\Temp	SUCCESS	Desired Access: Read Attributes, Di...
7:24:20...	7046e174e0f512...	5460	QueryBasicInformationFile	C:\Windows\Temp	SUCCESS	CreationTime: 5/7/2022 12:24:51 PM...
7:24:20...	7046e174e0f512...	5460	CloseFile	C:\Windows\Temp	SUCCESS	
7:24:20...	7046e174e0f512...	5460	CreateFile	C:\Windows\Temp	SUCCESS	Desired Access: Read Attributes, Di...
7:24:20...	7046e174e0f512...	5460	QueryBasicInformationFile	C:\Windows\Temp	SUCCESS	CreationTime: 5/7/2022 12:24:51 PM...
7:24:20...	7046e174e0f512...	5460	CloseFile	C:\Windows\Temp	SUCCESS	



4.3.2 Wireshark

Tổng cộng 1,473 packet được capture trong phiên thực thi. Filter DNS cho thấy 3 domain C2 được query ngay sau khi malware khởi chạy:

Domain	Thời điểm (s)	Xác nhận
baxe.pics	16.2	✓ VirusTotal: Malicious
coox.live	26.6	✓ VirusTotal: Malicious
zadno.run	36.8	✓ VirusTotal: Malicious

126	16.214447600	192.168.56.150	192.168.56.100	DNS	69 Standard query 0x2a2a A baxe.pics
127	16.223855300	192.168.56.100	192.168.56.150	DNS	85 Standard query response 0x2a2a A baxe.pics A 192.168.56.100

Hình : request domain baxe.pics

330	26.600797100	192.168.56.150	192.168.56.100	DNS	69 Standard query 0xb28d A coox.live
331	26.609042900	192.168.56.100	192.168.56.150	DNS	85 Standard query response 0xb28d A coox.live A 192.168.56.100
448	36.797007400	192.168.56.150	192.168.56.100	DNS	69 Standard query 0x2ec6 A zadno.run
449	36.805385600	192.168.56.100	192.168.56.150	DNS	85 Standard query response 0x2ec6 A zadno.run A 192.168.56.100

Hình : request 2 domain coox.live và zadno.run

Cả 3 domain đều được resolve về 192.168.56.100 (VMware gateway) do môi trường sandbox không có kết nối internet thật, C2 không reach được. Bên cạnh đó, traffic TCP đến port 28313 và 48261 lặp đều đặn với payload cố định 126 bytes xác nhận beacon pattern của SmokeLoader đang cố liên lạc về C2.

4.4 Chạy pipeline

```
← AUTOMATIC IOC EXTRACTION PIPELINE GUIDE ⓘ
Reviewed 2 files and considered Malware Validation

Ran forensic_report_from_validation - MALWARE (MCP Server)

Detecting and Summarizing IOCs

I have successfully executed the full 6-phase Volatility3 IOC extraction pipeline on your memory dump. Here is the step-by-step breakdown:

1. Phase 1 & 2: Discovery and OS Detection
  • Found multiple dumps and selected the most recently modified: /app/data/dumps/win_Smoke.raw (Modified: 2026-03-23)
  • Detected OS: Windows
  • Architecture: x64

2. Phase 3: Plugin Execution
  • Plugins Run: The full Windows plugin preset was executed.
  • Result: 18 plugins succeeded, 0 failed. Results were stored server-side successfully.

3. Phase 4: IOC Extraction
  • Total Extracted IOCs: 29
  • Top Dangerous Processes:
    ◦ MsMpEng.exe (PID: 3868) - HIGH Threat Score (0.95). Techniques: Code Injection (T1055), Malicious File/Hash (T1204).
  • The full structure output was successfully saved to /app/data/reports/ioc_extract_20260325T181848Z_rp_3ba5546c29134007.json.

4. Phase 5: IOC Validation
  • Validation Summary:
    ◦ Malicious: 19
    ◦ Suspicious: 3
    ◦ Benign: 7
  • Assessment: The threat level is HIGH with a 0.95 confidence score, suggesting a script-assisted in-memory loader.
  • Top Affected Process Findings: MsMpEng.exe (13 verified issues) and powershell.exe (3 verified issues).
  • (Note: Local heuristics only, as no API keys were configured for Threat Intel correlations).

5. Phase 6: Forensic Incident Report

The complete forensic Markdown report has been generated and saved based on the validated IOCs.

Final Deliverable Path:
/app/data/reports/forensic_incident_report_20260325T182046Z.md

Key Actionable Findings:

1. Critical systemic compromise (High Confidence) indicated by a script-assisted in-memory loader.
2. 17 distinct instances of process injection/hollowing were identified, primarily hijacking Windows Defender (MsMpEng.exe) for likely evasion purposes.
3. Execution pipelines involve abnormal powershell.exe artifacts executing malicious routines, combined with suspicious hashes being loaded.

G ⓘ 🔍 🗑️ Gemini 3.1 Pro (Preview) • 1x
```

Kết quả trả về:

Forensic Incident Report

```
- **Source validation file:**
/app/data/reports/ioc_validate_20260325T182030Z_dbba7fb0.json
- **Generated at:** 2026-03-25T18:20:46

## 1. Executive Summary

- **Malicious IOCs:** 19
- **Suspicious IOCs:** 3
```

```

- **Benign IOCs:** 7
- **Process injection regions:** 17 across 3 process(es)
- **Malicious hashes confirmed:** 0
- **Suspicious hashes:** 0
- **Persistence/evasion artifacts:** 3
- **Likely malware type:** script-assisted in-memory loader
- **Compromise level:** high

## 2. Host/System Profile

- OS type: windows
- OS version: unknown
- OS build: unknown
- Processor architecture: x64

## 3. Malware Actions Observed

### 3.1 Process Injection Actions (T1055)

**Summary:** **MsMpEng.exe** was targeted 13 time(s) (PID 3868);
**powershell.exe** was targeted 3 time(s) (PID 7812); **Wireshark.exe**
was targeted 1 time(s) (PID 6752).

| # | Target Process | PID | PPID | Start VPN | Region Size | Memory
Protection | Header / Hex (first 16 B) | MZ? | Source Plugin |
|---|-----|-----|-----|-----|-----|-----|-----|
| 1 | MsMpEng.exe | 3868 | 812 | 2853316526080 | 1.05 MB |
PAGE_EXECUTE_READWRITE | `56 57 53 55 41 54 41 55 41 56 41 57 48 83 ec 28`
| - | windows.malware.malfind.Malfind |
| 2 | MsMpEng.exe | 3868 | 812 | 2853321375744 | 1.05 MB |
PAGE_EXECUTE_READWRITE | `56 57 53 55 41 54 41 55 41 56 41 57 48 83 ec 28`
| - | windows.malware.malfind.Malfind |
| 3 | MsMpEng.exe | 3868 | 812 | 2853841076224 | 8.0 KB |
PAGE_EXECUTE_READWRITE | `55 48 8d 2c 24 48 83 ec 20 48 8b 01 48 8b 49 08`
| - | windows.malware.malfind.Malfind |
| 4 | MsMpEng.exe | 3868 | 812 | 2853841731584 | 4.0 KB |
PAGE_EXECUTE_READWRITE | `55 48 8d 2c 24 48 83 ec 20 48 8b 01 48 8b 49 08`
| - | windows.malware.malfind.Malfind |

```

```

| 5 | MsMpEng.exe | 3868 | 812 | 2853841993728 | 56.0 KB |
PAGE_EXECUTE_READWRITE | `55 48 8d 2c 24 48 83 ec 20 48 8b 01 48 8b 49 08`
| - | windows.malware.malfind.Malfind |
| 6 | MsMpEng.exe | 3868 | 812 | 2853994627072 | 8.0 KB |
PAGE_EXECUTE_READWRITE | `55 48 8d 2c 24 48 83 ec 20 48 8b 01 48 8b 49 08`
| - | windows.malware.malfind.Malfind |
| 7 | MsMpEng.exe | 3868 | 812 | 2853994299392 | 4.0 KB |
PAGE_EXECUTE_READWRITE | `55 48 8d 2c 24 48 83 ec 20 48 8b 01 48 8b 49 08`
| - | windows.malware.malfind.Malfind |
| 8 | MsMpEng.exe | 3868 | 812 | 2853994233856 | 4.0 KB |
PAGE_EXECUTE_READWRITE | `55 48 8d 2c 24 48 83 ec 20 48 8b 01 48 8b 49 08`
| - | windows.malware.malfind.Malfind |
| 9 | MsMpEng.exe | 3868 | 812 | 2853995282432 | 4.0 KB |
PAGE_EXECUTE_READWRITE | `55 48 8d 2c 24 48 83 ec 20 48 8b 01 48 8b 49 08`
| - | windows.malware.malfind.Malfind |
| 10 | MsMpEng.exe | 3868 | 812 | 2853995216896 | 12.0 KB |
PAGE_EXECUTE_READWRITE | `55 48 8d 2c 24 48 83 ec 20 48 8b 01 48 8b 49 08`
| - | windows.malware.malfind.Malfind |
| 11 | MsMpEng.exe | 3868 | 812 | 2854000984064 | 1.00 MB |
PAGE_EXECUTE_READWRITE | `cc cc cc cc cc cc cc cc cc cc cc cc cc cc cc cc`
| - | windows.malware.malfind.Malfind |
| 12 | MsMpEng.exe | 3868 | 812 | 2854002032640 | 2.00 MB |
PAGE_EXECUTE_READWRITE | `cc cc cc cc cc cc cc cc cc cc cc cc cc cc cc cc`
| - | windows.malware.malfind.Malfind |
| 13 | MsMpEng.exe | 3868 | 812 | 2854005506048 | 4.0 KB |
PAGE_EXECUTE_READWRITE | `55 48 8d 2c 24 48 83 ec 20 48 8b 01 48 8b 49 08`
| - | windows.malware.malfind.Malfind |
| 14 | Wireshark.exe | 6752 | 4732 | 2132258652160 | 64.0 KB |
PAGE_EXECUTE_READWRITE | `00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00`
| - | windows.malware.malfind.Malfind |
| 15 | powershell.exe | 7812 | 4732 | 2797947977728 | 52.0 KB |
PAGE_EXECUTE_READWRITE | `00 00 00 00 00 00 00 00 90 78 1e 73 8b 02 00 00`
| - | windows.malware.malfind.Malfind |
| 16 | powershell.exe | 7812 | 4732 | 138490051690496 | 64.0 KB |
PAGE_EXECUTE_READWRITE | `00 00 00 00 00 00 00 00 78 0d 00 00 00 00 00 00`
| - | windows.malware.malfind.Malfind |
| 17 | powershell.exe | 7812 | 4732 | 138490051756032 | 640.0 KB |
PAGE_EXECUTE_READWRITE | `d8 ff ff ff ff ff ff ff 08 00 00 00 00 00 00`
| - | windows.malware.malfind.Malfind |

```

3.2 Persistence & Suspicious Actions

#	Artifact Type	Value	Binary / Path	Associated Technique
Detecting Plugin				
--- ----- ----- ----- ----- -----				

1	Service	MDCoreSvc	"C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26010.5-0\MpDefenderCoreService.exe"	T1543.003 (Windows Service) windows.svcscan.SvcScan
2	Service	WinDefend	"C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.26010.5-0\MsMpEng.exe"	T1543.003 (Windows Service) windows.svcscan.SvcScan
3	Filepath	a:\uf8e9;\u4912\u2492\uc180\u08db\ub906;\ub76c;\ub230;\ub0bb;\ua72b;\u9655;\u9f19;\u9cc5\u1a51\ucc0a\u3b99\uc800 - T1036 (Masquerading) windows.malware.ldrmodules.LdrModules		

3.3 Confirmed Malicious Hashes

No malicious hashes confirmed.

4. MITRE ATT&CK Techniques Observed

- ****T1036 (Masquerading):**** Evidenced by Obfuscated path: a:\uf8e9;\u4912\u2492\uc180\u08db\ub906;\ub76c;\ub230;\ub0bb.
- ****T1055 (Process Injection):**** Evidenced by PAGE_EXECUTE_READWRITE anomaly in MsMpEng.exe (PID 3868); PAGE_EXECUTE_READWRITE anomaly in Wireshark.exe (PID 6752); PAGE_EXECUTE_READWRITE anomaly in powershell.exe (PID 7812).
- ****T1055.001****
- ****T1543.003 (Windows Service):**** Evidenced by Service 'MDCoreSvc' → "C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.260; Service 'WinDefend' → "C:\ProgramData\Microsoft\Windows Defender\Platform\4.18.260.

5. Conclusion

****13 PAGE_EXECUTE_READWRITE memory injection region(s)**** detected in `MsMpEng.exe` and 2 other process(es) – consistent with reflective DLL injection or shellcode staging (T1055). ****Persistence via Windows**

```
service(s)** (`MDCoreSvc`, `WinDefend`) detected – review binary paths for tampering (T1543.003).
```

****Recommended actions:****

1. Dump memory of PID(s) 3868, 6752, 7812 for deeper analysis.
3. Review and disable suspicious services listed in Section 3.2.

4.5 Phân tích kết quả và đối chiếu

Network IOC:

```
"network_iocs": [  
  {  
    "type": "ipv4",  
    "category": "network",  
    "value": "192.168.56.100",  
    "confidence": 0.7,  
    "source": "windows.netscan.NetScan",  
    "context": {  
      "process": "",  
      "pid": "",  
      "remote_port": 443,  
      "local_port": 55343,  
      "state": "ESTABLISHED",  
      "proto": "TCPv4",  
      "reasons": [  
        "established",  
        "beaconing_pattern"  
      ],  
      "is_private": true,  
      "is_lab_network": true,  
      "technique": "T1071",  
      "category": "network",  
      "beaconing": true,  
      "connection_count": 8  
    },  
    "extracted_at": "2026-03-25T18:18:48.576942"  
  }  
]
```


Phát hiện được sự kết nối giữa C2 Server và máy người dùng, ở đây malware đã dùng kỹ thuật “obfuscation” để che dấu dấu vết nên chúng ta sẽ không thấy được tập tin thực thi

TÀI LIỆU THAM KHẢO

Bromiley, M. (2019). *Windows memory forensics: Detecting malware injection techniques*. SANS Institute.

<https://www.sans.org/white-papers/windows-memory-forensics/>

Hasherezade. (2021). *SmokeLoader analysis*. MalwareBazaar.

<https://bazaar.abuse.ch/sample/7046e174e0f51284f119a0061ad816e066a5f5b24c775f8cfcd613240c4a6912/>

Ligh, M. H., Case, A., Levy, J., & Walters, A. (2014). *The art of memory forensics: Detecting malware and threats in Windows, Linux, and Mac memory*. Wiley.

Microsoft Corporation. (2023). *Antimalware Scan Interface (AMSI)*. Microsoft Learn.

<https://learn.microsoft.com/en-us/windows/win32/amsi/antimalware-scan-interface-portal>

MITRE ATT&CK. (2023). *SmokeLoader (S0226)*. MITRE Corporation.

<https://attack.mitre.org/software/S0226/>

MITRE ATT&CK. (2023). *T1055 – Process injection*. MITRE Corporation.

<https://attack.mitre.org/techniques/T1055/>

MITRE ATT&CK. (2023). *T1070.006 – Timestomp*. MITRE Corporation.

<https://attack.mitre.org/techniques/T1070/006/>

MITRE ATT&CK. (2023). *T1071.001 – Application layer protocol: Web protocols*. MITRE Corporation.

<https://attack.mitre.org/techniques/T1071/001/>

MITRE ATT&CK. (2023). *T1112 – Modify registry*. MITRE Corporation.

<https://attack.mitre.org/techniques/T1112/>

MITRE ATT&CK. (2023). *T1497 – Virtualization/sandbox evasion*. MITRE Corporation.

<https://attack.mitre.org/techniques/T1497/>

Russinovich, M., & Garnier, T. (2023). *Process Monitor v3.92*. Microsoft Sysinternals.

<https://learn.microsoft.com/en-us/sysinternals/downloads/procmon>

Volatility Foundation. (2023). *Volatility 3 framework documentation*.

<https://volatility3.readthedocs.io/>

Wireshark Foundation. (2023). *Wireshark user's guide (v4.x)*.

https://www.wireshark.org/docs/wsug_html_chunked/

VirusTotal. (2026, March 21). *Analysis of coox.live*. Google LLC.

<https://www.virustotal.com/gui/domain/coox.live>

VirusTotal. (2026, March 21). *Analysis of baxe.pics*. Google LLC.

<https://www.virustotal.com/gui/domain/baxe.pics>

VirusTotal. (2026, March 21). *Analysis of zadno.run*. Google LLC.

<https://www.virustotal.com/gui/domain/zadno.run>